

OnePass / OneFitPass

Final Project Report

Course: CSIS 3275 – 070, Software Engineering

Instructor: Caesar Jude Clemente

Team Members: Mikhail Senatorov, Jana Paul, Omkar Patel,

Submission Type: Final Project Documentation integrating Progress Report 1, Progress Report 2, and Final Implementation Review

Repository Reviewed: OnePass (Spring Boot MVC implementation)

Date: April 2026

Abstract

This final report presents the complete academic and technical documentation for **OnePass**, also referred to in the earlier business and design documents as **OneFitPass**. The report consolidates the work completed across **Progress Report 1** and **Progress Report 2** and aligns it with the **actual implemented software product** contained in the project repository. The original concept envisioned a broader digital fitness access ecosystem with a mobile application, QR-based gym entry, partner gym onboarding, cloud-hosted infrastructure, and payment integration. During implementation, the project scope was intentionally reduced to a manageable minimum viable product (MVP) that could be completed, tested, and demonstrated within the academic term. The resulting solution is a working Spring Boot web application that supports user registration, authentication, membership-tier management, gym selection, gym check-in processing, check-in history, account management, and administrator oversight.

The report has two central purposes. First, it documents the **business and engineering rationale** behind the project, including the market opportunity, personas, revenue model, feasibility study, and system requirements that were developed during the analysis phase. Second, it provides a **design and implementation record** of the final delivered system, including architecture, database design, user interface design, UML-style diagrams, security measures, testing strategy, deployment instructions, and KPI evaluation. Where appropriate, the document explicitly distinguishes between the **planned platform vision** described in the earlier progress reports and the **implemented**

repository state reviewed for the final submission. This distinction is important because it allows the report to remain academically honest while still showing continuity between the problem domain, the design phase, and the final product.

From a software engineering perspective, the implemented MVP demonstrates a coherent layered MVC architecture using **Java 17, Spring Boot 3.4.4, Spring MVC, Thymeleaf, Spring Data JPA, jBCrypt**, and an **H2 in-memory relational database**. The repository contains **18 main Java source files, 5 test classes, 9 HTML templates including one shared navigation fragment, and one shared CSS stylesheet of approximately 719 lines**. Automated testing confirms that the application context loads successfully and that unit and controller behaviors work as expected, with **23 tests passing successfully**. The project therefore satisfies the primary academic goal of producing a working, demonstrable software system while also preserving a strategic roadmap for future expansion into payment workflows, QR-based access, production persistence, search/filtering, and cloud deployment.

Document Integration Note

This final report integrates material from three stages of the project lifecycle:

1. **Progress Report 1** — business case, market gap, revenue model, feasibility study, and requirements analysis.
2. **Progress Report 2** — design goals, architecture, database design, interface design, UML-oriented solution design, and design-phase KPIs.
3. **Final Repository Review** — the actual implemented Spring Boot MVC product, tested locally and documented from the codebase state reviewed in April 2026.

Because the final implementation is narrower than the originally proposed platform, this report uses the following convention:

- **Planned / Proposed** refers to ideas or capabilities described in earlier reports but not fully implemented in the repository.
- **Implemented / Delivered** refers only to behavior confirmed by repository inspection and local test execution.

This distinction ensures consistency, traceability, and professional accuracy.

Index

Click any section to jump directly to it.

1. [1. Introduction](#)
 1. [1.1 Project Background](#)
 2. [1.2 Purpose of This Final Report](#)
 3. [1.3 Scope of the Delivered System](#)
 4. [1.4 Methodology Used for Final Documentation](#)
 5. [1.5 Structure of the Report](#)
2. [2. Business Case and Strategic Context](#)
 1. [2.1 Executive Summary of the Business Case](#)
 2. [2.2 Business Goals](#)
 3. [2.3 Proposed Value Proposition](#)
 1. [For Users](#)
 2. [For Gym Partners](#)
 3. [For Administrators / Platform Operators](#)
 4. [2.4 Business Case Summary Table](#)
 5. [2.5 Strategic Interpretation of the Final MVP](#)
3. [3. Problem Statement, Stakeholders, and Market Gap](#)
 1. [3.1 Problem Statement](#)
 2. [3.2 Stakeholders](#)
 3. [3.3 Key Personas](#)
 1. [3.3.1 Urban Professional](#)
 2. [3.3.2 Student or Young Adult](#)
 3. [3.3.3 Frequent Traveler or Remote Worker](#)
 4. [3.3.4 Platform Administrator](#)
 4. [3.4 Market Background](#)
 5. [3.5 Unmet Needs and Current Limitations](#)

- 6. 3.6 Competitor Context
- 7. 3.7 Comparative Positioning Table
- 8. 3.8 Planned vs Implemented Problem Coverage
- 4. 4. Revenue Model and Business Viability
 - 1. 4.1 Revenue Model Concept
 - 2. 4.2 Subscription Tier Concept from the Business Plan
 - 1. Original Business-Oriented Example Tiers
 - 2. Implemented MVP Membership Tiers
 - 3. 4.3 Effective Cost per Visit
 - 4. 4.4 Gym Partner Revenue Logic in the Planned Model
 - 5. 4.5 ARR Projection and Growth Horizon
 - 6. 4.6 Revenue Projection Example
 - 7. 4.7 Financial Interpretation of the MVP
 - 8. 4.8 Commercial Risks
- 5. 5. Feasibility Study
 - 1. 5.1 Purpose of the Feasibility Study
 - 2. 5.2 Market Feasibility
 - 3. 5.3 Technical Feasibility
 - 1. 5.3.1 Feasibility in the Original Plan
 - 2. 5.3.2 Feasibility in the Implemented MVP
 - 4. 5.4 Operational Feasibility
 - 5. 5.5 Economic Feasibility
 - 6. 5.6 Schedule Feasibility
 - 7. 5.7 Legal and Compliance Feasibility
 - 8. 5.8 Feasibility Summary Table
 - 9. 5.9 Final Feasibility Conclusion
- 6. 6. Requirements Engineering and Analysis
 - 1. 6.1 Sources of Requirements

2. 6.2 Requirement Categories
3. 6.3 Functional Requirements from the Larger Vision
4. 6.4 Functional Requirements Traceability Table
5. 6.5 Implemented Functional Scope Summary
6. 6.6 Non-Functional Requirements
 1. 6.6.1 Usability
 2. 6.6.2 Maintainability
 3. 6.6.3 Testability
 4. 6.6.4 Security
 5. 6.6.5 Portability
 6. 6.6.6 Performance and Scalability
7. 6.7 Non-Functional Requirement Mapping Table
8. 6.8 Use Cases Covered by the MVP
9. 6.9 Requirement Prioritization (MoSCoW View)
10. 6.10 Requirements Conclusion
7. 7. Design Goals and Scope Realignment
 1. 7.1 Role of Design Goals
 2. 7.2 Primary Design Goals
 3. 7.3 Design Goal Mapping Table
 4. 7.4 Why the Implemented Architecture Differs from the Planned Architecture
 5. 7.5 Design Constraints in the MVP
 6. 7.6 Design Philosophy of the Final Product
8. 8. System Architecture and Technology Stack
 1. 8.1 Architectural Style
 2. 8.2 Layered Architecture Overview
 3. 8.3 Technology Stack Table
 4. 8.4 Repository Structure
 5. 8.5 Architectural Narrative

6. 8.6 Configuration Highlights
7. 8.7 Seeded Data Architecture
8. 8.8 Architectural Strengths
9. 8.9 Architectural Limitations
9. 9. Hardware and Software Configuration
 1. 9.1 Purpose of the Configuration Section
 2. 9.2 Software Requirements
 3. 9.3 Hardware Requirements
 4. 9.4 Development Environment Characteristics
 5. 9.5 Runtime Environment
 6. 9.6 Production Readiness Consideration
10. 10. Database Design
 1. 10.1 Database Choice
 2. 10.2 Core Entities
 3. 10.3 Entity Descriptions
 1. 10.3.1 User
 2. 10.3.2 Gym
 3. 10.3.3 CheckIn
 4. 10.3.4 MembershipType
 4. 10.4 Table Design
 1. 10.4.1 Table: `users`
 2. 10.4.2 Table: `gyms`
 3. 10.4.3 Table: `check\_ins`
 5. 10.5 Entity-Relationship Diagram
 6. 10.6 Normalization Discussion
 1. First Normal Form (1NF)
 2. Second Normal Form (2NF)
 3. Third Normal Form (3NF)

- 7. 10.7 Persistence Behavior
- 8. 10.8 Repository Methods and Query Behavior
- 9. 10.9 Database Design Assessment
- 11. 11. Interface Design
 - 1. 11.1 Interface Design Philosophy
 - 2. 11.2 Presentation Architecture
 - 3. 11.3 Shared Navigation Design
 - 4. 11.4 Screen Inventory
 - 5. 11.5 Login and Registration Pages
 - 6. 11.6 Home Dashboard
 - 7. 11.7 Check-in Page
 - 8. 11.8 History Page
 - 9. 11.9 Account Page
 - 10. 11.10 Membership Page
 - 11. 11.11 Admin Page
 - 12. 11.12 Styling and Responsiveness
 - 13. 11.13 UI Design Strengths
 - 14. 11.14 UI Design Limitations
- 12. 12. UML and System Diagrams
 - 1. 12.1 Purpose of the Diagram Section
 - 2. 12.2 Use Case Overview
 - 1. Main Actors
 - 2. Supported User Use Cases
 - 3. Supported Admin Use Cases
 - 4. Deferred Use Cases
 - 3. 12.3 Textual Use Case Diagram
 - 4. 12.4 Class Diagram
 - 5. 12.5 Mermaid Version of the Class Diagram

6. 12.6 Sequence Diagram — Authentication Flow
7. 12.7 Sequence Diagram — Registration Flow
8. 12.8 Sequence Diagram — Check-In Flow
9. 12.9 Sequence Diagram — Membership Change
10. 12.10 Activity Diagram — User Journey
11. 12.11 Component Diagram
12. 12.12 Diagram Interpretation
13. 13. Final Implemented Software Product
 1. 13.1 Delivered Scope Overview
 2. 13.2 Application Entry Point
 3. 13.3 Controller Layer Summary
 4. 13.4 Service Layer Summary
 5. 13.5 Repository Layer Summary
 6. 13.6 Core Domain Logic
 1. 13.6.1 Registration
 2. 13.6.2 Authentication
 3. 13.6.3 Check-In Business Rules
 4. 13.6.4 History Retrieval
 5. 13.6.5 Account Management
 7. 13.7 Membership Tier Design
 8. 13.8 Seeded Demo Data
 1. 13.8.1 Seeded Gyms
 2. 13.8.2 Seeded Accounts
 9. 13.9 Codebase Summary Metrics
 10. 13.10 Strength of the Delivered Product
 11. 13.11 Deferred Scope
 12. 13.12 Interpretation of MVP Completeness
14. 14. Security Design and Implementation

1. 14.1 Security Scope in the MVP
2. 14.2 Authentication Mechanism
3. 14.3 Authorization
4. 14.4 Password Security
5. 14.5 Input Validation
6. 14.6 Security-Relevant Routes
7. 14.7 Session Handling
8. 14.8 Security Strengths
9. 14.9 Security Limitations
10. 14.10 Future Security Hardening Roadmap
11. 14.11 Security Assessment
15. 15. Testing Design and Testing Results
 1. 15.1 Testing Objectives
 2. 15.2 Testing Techniques Used
 3. 15.3 Test Classes
 4. 15.4 Verified Test Execution
 5. 15.5 Per-Class Results
 6. 15.6 What the Tests Prove
 7. 15.7 Warnings Observed During Test Execution
 8. 15.8 Testing Gaps
 9. 15.9 Quality Assessment of the Test Strategy
 10. 15.10 Suggested Future Testing Enhancements
16. 16. Deployment, Operations, and Usage
 1. 16.1 Deployment Philosophy
 2. 16.2 Prerequisites
 3. 16.3 Startup Steps
 1. Windows Command
 2. macOS / Linux Command

4. 16.4 Demo Accounts
5. 16.5 H2 Console Access
6. 16.6 Operational Notes
7. 16.7 Demonstration Script Suggestion
8. 16.8 Operations Gaps for Production Use
17. 17. KPI Evaluation
 1. 17.1 KPI Context
 2. 17.2 Requirement Analysis Phase KPIs
 1. Assessment
 3. 17.3 Design Phase KPIs
 4. 17.4 Implementation KPIs
 5. 17.5 KPI Summary
 6. 17.6 KPI Interpretation
18. 18. Risk Management, Limitations, and Future Roadmap
 1. 18.1 Risk Register Overview
 2. 18.2 Current Limitations of the Repository
 3. 18.3 Planned vs Implemented Scope Reconciliation
 4. 18.4 Roadmap for Version 2
 1. Short-Term Enhancements
 2. Medium-Term Enhancements
 3. Long-Term Enhancements
 5. 18.5 Production Transition Considerations
19. 19. Lessons Learned and Reflection
 1. 19.1 Importance of Scope Control
 2. 19.2 Value of Layered Architecture
 3. 19.3 Domain Logic Before Infrastructure
 4. 19.4 Testing as a Confidence Tool
 5. 19.5 Honest Documentation Matters

20. 20. Conclusion

21. 21. References

22. 22. Appendices

1. Appendix A — Review Checklist
2. Appendix B — Requirement Traceability Snapshot
3. Appendix C — Route Inventory
4. Appendix D — Suggested Formatting for Submission
5. Appendix E — Glossary
6. Appendix F — Detailed Screen-by-Screen Walkthrough
 1. F.1 Login Screen
 2. F.2 Registration Screen
 3. F.3 Home Dashboard Screen
 4. F.4 Check-In Screen
 5. F.5 History Screen
 6. F.6 Account Screen
 7. F.7 Membership Screen
 8. F.8 Admin Dashboard Screen
7. Appendix G — Detailed Technical Documentation by Module
 1. G.1 Application Bootstrap and Configuration
 2. G.2 DataLoader
 3. G.3 AuthController
 4. G.4 HomeController
 5. G.5 CheckInController
 6. G.6 HistoryController
 7. G.7 AccountController
 8. G.8 AdminController
 9. G.9 UserService
 10. G.10 CheckInService

11. G.11 GymService
12. G.12 UserRepository
13. G.13 GymRepository
14. G.14 CheckInRepository
15. G.15 User Model
16. G.16 Gym Model
17. G.17 CheckIn Model
18. G.18 MembershipType Enum
8. Appendix H — Planned vs Implemented Scope Reconciliation Matrix
 1. H.1 Strategic Interpretation
 2. H.2 Comparison Matrix
 3. H.3 Why the Scope Reduction Was Reasonable
 4. H.4 Scope Reduction as an Engineering Success
 5. H.5 What the Current Repository Proves
 6. H.6 What the Repository Does Not Yet Prove
9. Appendix I — Representative Acceptance and Test Catalog
 1. I.1 Registration Acceptance Cases
 2. I.2 Authentication Acceptance Cases
 3. I.3 Home Dashboard Acceptance Cases
 4. I.4 Check-In Acceptance Cases
 5. I.5 History Acceptance Cases
 6. I.6 Account and Membership Acceptance Cases
 7. I.7 Admin Acceptance Cases
 8. I.8 Deployment and Demo Acceptance Cases
 9. I.9 Test Coverage Interpretation
 10. I.10 Recommended Future Acceptance Expansion
10. Appendix J — Expanded Non-Functional Requirement Interpretation and Design Mapping
 1. J.1 Usability as a Design Driver

2. J.2 Maintainability as a Structural Goal
3. J.3 Portability and Demo Readiness
4. J.4 Testability and Verification
5. J.5 Security as a Controlled MVP Concern
6. J.6 Reliability and Data Integrity
7. J.7 Scalability and Performance Expectations
8. J.8 Compatibility and Browser Accessibility
9. J.9 Non-Functional Requirement Mapping Matrix
10. J.10 Lessons from the Non-Functional View
11. Appendix K — Extended Business Process Narratives
 1. K.1 Account Registration Process Narrative
 2. K.2 Authentication Process Narrative
 3. K.3 Dashboard Access Process Narrative
 4. K.4 Gym Check-In Process Narrative
 5. K.5 Visit History Process Narrative
 6. K.6 Profile Update Process Narrative
 7. K.7 Membership Change Process Narrative
 8. K.8 Admin Monitoring Process Narrative
 9. K.9 Failed and Edge-Case Process Narrative
 10. K.10 Business Process Reflection
12. Appendix L — Future Production Architecture Proposal
 1. L.1 Evolution Strategy Overview
 2. L.2 Recommended Persistence Evolution
 3. L.3 Recommended Security Evolution
 4. L.4 Recommended Deployment Evolution
 5. L.5 Recommended API Strategy
 6. L.6 Recommended Gym and Partner Expansion
 7. L.7 Recommended Billing Evolution

8. L.8 Recommended Event and Analytics Evolution
9. L.9 Suggested Future Architecture Diagram
10. L.10 Why the MVP Still Matters in This Future View
11. L.11 Migration Path Summary Table
12. L.12 Final Architectural Reflection

1. Introduction

1.1 Project Background

The Canadian fitness industry offers a large number of independent gyms, regional chains, premium clubs, and budget facilities. Although this variety creates consumer choice, it also creates fragmentation. Individuals who live, work, or study in multiple areas often need access to different facilities at different times of the day. Under conventional gym membership models, such users must either commit to one location and accept reduced flexibility or pay for multiple memberships and carry the combined cost burden. This creates a clear usability and affordability problem, especially for urban professionals, students, and mobile workers.

The OneFitPass idea emerged as a response to this fragmentation. The original platform vision proposed a unified digital pass that would allow users to subscribe once and gain access to multiple partner gyms. Over time, the business vision expanded to include several supporting capabilities: QR-based check-in, visit validation, partner-gym onboarding, payout calculations, subscription billing, reporting, and possibly a cross-platform mobile application. These elements shaped the early analysis and design phases documented in Progress Reports 1 and 2.

For the final phase of the course project, the team implemented a focused MVP named **OnePass**. Rather than attempting to deliver every planned feature, the team concentrated on the most essential user journey: account creation, membership management, gym check-in, visit history, and basic administration. This decision allowed the project to achieve a working software outcome with demonstrable engineering quality.

1.2 Purpose of This Final Report

The purpose of this report is to provide a single consolidated document that:

- explains the business problem and strategic motivation for the platform;
- documents the requirements and feasibility analysis from the early stages of the project;
- presents the design goals and architectural decisions made during system design;
- describes the actual implemented system found in the repository;

- evaluates the final product using testing evidence, KPI measurements, and implementation scope;
- identifies limitations, deferred features, and a realistic future roadmap.

This final report is therefore both a **project history** and a **technical product record**.

1.3 Scope of the Delivered System

The delivered repository contains a **Spring Boot MVC web application** running on **localhost:8080** with an **H2 in-memory database** and seeded demo data. The MVP supports:

- user registration with membership-tier selection;
- login and logout using session-based authentication;
- membership tiers (Bronze, Silver, Gold);
- gym listing and gym check-in;
- business rules for daily visit limits and same-gym cooldown;
- reverse-chronological history display;
- account profile update and password change;
- membership change;
- administrator dashboard with user and system summaries.

The delivered system does **not** include payment processing, partner-gym self-service onboarding, QR-based entry, history searching/filtering, a mobile client, or cloud deployment.

1.4 Methodology Used for Final Documentation

This report was produced through a combination of:

1. Review of **Progress Report 1** for business case, feasibility, and requirements material.
2. Review of **Progress Report 2** for design-phase expectations and architectural intent.
3. Direct repository inspection of source code, templates, configuration files, and tests.
4. Verification of runtime and testing behavior through Maven test execution.
5. Synthesis of the above materials into a coherent final academic report.

The repository inspection verified the following concrete technical facts:

- Java version: 17
- Spring Boot version: 3.4.4
- H2 in-memory database enabled
- server port: 8080
- main Java files: 18
- test classes: 5
- HTML templates: 9 including one shared fragment
- CSS files: 1
- automated tests passing: 23/23

1.5 Structure of the Report

Sections 2 through 6 consolidate the **analysis-phase work** from Report 1. Sections 7 through 12 consolidate the **design-phase work** from Report 2 and align it with the final implementation. Sections 13 through 20 document the **implemented software product**, including security, testing, deployment, KPIs, limitations, and future direction. Appendices provide review tables, traceability matrices, and supporting reference material.

2. Business Case and Strategic Context

2.1 Executive Summary of the Business Case

OneFitPass was initially conceived as a digital access platform for traditional gyms in . The business idea is simple: instead of purchasing multiple separate memberships, a user would subscribe once and gain access to a network of participating gyms. This model attempts to solve three major market inefficiencies at the same time.

First, it addresses the user-side problem of **fragmented access**. A single-gym subscription works well only for users with stable routines and geographically fixed schedules. In practice, many people split time between home, school, work, family commitments, and travel. A subscription that provides access to only one facility becomes inefficient in such conditions.

Second, the platform creates value for **partner gyms**, especially smaller or mid-sized facilities that experience off-peak underutilization. Rather than investing heavily in marketing to capture entirely new subscribers, a gym could participate in a shared access platform and receive incremental traffic and revenue from verified visits.

Third, the concept presents an opportunity for a **platform business model** that aligns consumer convenience with gym utilization. By acting as infrastructure for access, validation, and reporting, OneFitPass could position itself between standalone gym memberships and class-centric marketplaces such as ClassPass.

In the academic project context, the final implementation did not attempt to validate the full commercial business model end-to-end. Instead, it validated the **core product hypothesis**: that membership logic, access rules, gym selection, account management, and administrative oversight can be implemented as a coherent software system. The final product therefore serves as a functioning MVP and proof-of-concept for the larger business vision.

2.2 Business Goals

The original business goals presented in Progress Report 1 can be summarized as follows:

1. **Reduce barriers to fitness participation** by making access more flexible and more affordable than maintaining multiple memberships.
2. **Create a location-agnostic training model** for users who move across neighborhoods, campuses, or business districts.
3. **Deliver supplementary revenue to partner gyms**, especially during low-occupancy periods.
4. **Establish a scalable recurring revenue platform** through subscription pricing, commissions, and later value-added partnerships.
5. **Reach approximately \$1,000,000 in Annual Recurring Revenue (ARR)** within a 3–4 year growth horizon under a larger commercial rollout scenario.

These goals remain strategically relevant even though the implemented repository is narrower than the original business plan.

2.3 Proposed Value Proposition

For Users

- One account instead of multiple gym subscriptions
- Flexible access across locations
- Simpler tracking of visits and plan status
- Lower effective cost per visit when usage is high
- Easier onboarding and account management

For Gym Partners

- Incremental traffic without building a separate digital platform
- Better utilization of off-peak capacity
- Shared demand generation through a central platform
- Future visibility into check-in patterns and payouts

For Administrators / Platform Operators

- Centralized visibility of users and system usage
- Standardized plan logic and access rules
- Expandable foundation for compliance, billing, and reporting

2.4 Business Case Summary Table

Business Dimension	Core Opportunity	Relevance to Final MVP
User convenience	Reduce the burden of multiple memberships	Demonstrated through one-account, multi-gym interaction
Cost efficiency	Lower effective access cost through tiered plans	Represented by membership tiers and cost-per-visit dashboard metric
Gym monetization	Generate incremental revenue for partners	Conceptual only in MVP; no partner payout workflow implemented
Platform scalability	Grow through subscriptions and partnerships	Partially demonstrated through modular architecture
Market differentiation	Focus on traditional gym access, not just classes	Preserved as the central product identity

2.5 Strategic Interpretation of the Final MVP

The final repository should be understood as a **strategic subset** of the original business concept. It does not prove the complete commercial viability of the future platform, but it does establish the following foundational truths:

- account-based access control can be modeled and managed;
- membership policies can be formalized in software;
- gym visits can be validated against business rules;
- a usable web interface can support the primary user journey;
- the system is testable, deployable locally, and ready for further extension.

In other words, the MVP delivers the operational skeleton of the larger OneFitPass idea.

3. Problem Statement, Stakeholders, and Market Gap

3.1 Problem Statement

The central problem addressed by OneFitPass is the **fragmentation of fitness access** in . Users who need flexibility across multiple neighborhoods or cities often face a poor choice set: commit to one gym and lose convenience, or maintain several memberships and pay far more than is efficient. Drop-in visits partially solve the problem but quickly become expensive and do not offer continuity, history tracking, or account-level benefits.

From a systems perspective, the existing market lacks a user-friendly and accessible mechanism for **unscheduled, multi-location, traditional gym usage** under one account. While class-oriented marketplaces and employer wellness programs exist, they do not fully address the needs of frequent independent gym users.

The problem also affects facility operators. Smaller gyms may have available capacity during large portions of the day but lack a cost-effective channel to convert that unused space into predictable, trackable demand. A platform that can route members into unused inventory—while maintaining verification and access control—could benefit both consumers and facilities.

3.2 Stakeholders

The project involves the following main stakeholder groups:

Stakeholder	Role in the System	Main Interests
Gym users	End users of the platform	Flexible access, affordability, simplicity
Partner gyms	Service providers in the network	Reliable check-ins, payouts, utilization
Platform administrators	Operators of the system	Oversight, metrics, user management
Development team	Designers and implementers	Deliverable scope, architecture quality, maintainability
Course instructor / evaluators	Academic reviewers	Sound engineering process, traceability, working product

3.3 Key Personas

3.3.1 Urban Professional

The urban professional is typically between 25 and 45 years old and frequently moves between home, work, meetings, and transit hubs. This user values time efficiency and location flexibility. A single-location gym membership is not ideal because it forces the user to plan fitness around geography rather than schedule. In the OneFitPass concept, this persona benefits from the ability to train near the office one day and near home on another day.

3.3.2 Student or Young Adult

This persona is budget-conscious and often uncertain about long-term commitments. Students and younger users are interested in trying different gyms and adapting their schedule around classes, work, and transportation. High joining fees, annual commitments, and multiple subscription costs create barriers for this group. A tiered, flexible system is attractive because it offers variety without the cost of several separate memberships.

3.3.3 Frequent Traveler or Remote Worker

This persona may work from several locations or travel regularly between neighborhoods or cities. Traditional membership structures perform poorly for this user because geographic access is

limited and drop-in pricing is expensive. A multi-location access model is particularly compelling here because consistency matters more than loyalty to one physical gym branch.

3.3.4 Platform Administrator

Although less emphasized in the consumer-facing business case, the administrator persona is important in software design. Administrators need visibility into users, system activity, and operational status. In the final MVP, this persona is represented by an admin dashboard that provides a simple overview of registered users and total check-ins.

3.4 Market Background

Progress Report 1 identified the Canadian fitness industry as a strong context for a platform of this type. The report cited approximately **9,395 gym, health, and fitness club businesses** in and estimated industry revenue of approximately **\$5.8 billion**. It also noted that adult membership penetration remains limited relative to total population, which suggests both opportunity and barriers. Cost, inflexibility, and fragmented service models remain significant factors limiting broader participation.

The project's initial geographic focus was **Metro Vancouver**, a reasonable launch market because it combines high population density, multiple gym options, and user lifestyles that often require movement across different neighborhoods.

3.5 Unmet Needs and Current Limitations

The early analysis identified several unmet needs in the market:

1. **Need for walk-in flexibility** without advance class booking.
2. **Need for traditional gym access**, not only boutique classes or scheduled sessions.
3. **Need for transparent pricing**, rather than opaque credit systems or employer-only packages.
4. **Need for a unified user experience** with history, membership status, and account management.
5. **Need for access verification and anti-abuse controls**, such as cooldowns and visit limits.

These needs are directly reflected in the MVP. Even though the final system does not implement payments or QR entry, it does implement the essential logic of flexible access and rule-based visit control.

3.6 Competitor Context

Progress Report 1 positioned OneFitPass against several types of alternatives:

- **Single-chain gym memberships** such as GoodLife Fitness, Anytime Fitness, or Fit4Less.
- **ClassPass**, which is credit-based and class-oriented.
- **Wellhub (Gympass)**, which is heavily employer-focused.

The key distinction is that OneFitPass targets **consumer-driven access to traditional gym facilities**. This is an important strategic identity because it positions the platform as access infrastructure rather than as a class marketplace.

3.7 Comparative Positioning Table

Aspect	ClassPass	Wellhub / Gympass	Single-Gym Chains	OneFitPass Concept
Main focus	Classes and boutique sessions	Corporate wellness access	Access to one chain network	Flexible access to multiple traditional gyms
User model	Individual consumer	Employer-sponsored	Individual consumer	Individual consumer, later potentially hybrid
Usage pattern	Scheduled booking	Employer plan dependent	Chain-limited access	Walk-in or spontaneous access
Pricing model	Credit-based	Employer plan tiers	Monthly gym fee	Tiered subscription
Traditional gym focus	Limited	Mixed	Yes, but chain-only	Yes
Multi-gym flexibility	Partial	Partial, employer-dependent	Only within chain	Core value proposition

3.8 Planned vs Implemented Problem Coverage

The original concept proposed a complete operational solution including QR validation, payments, and partner workflows. The implemented MVP covers a narrower but still meaningful subset of the problem space:

Problem Area	Planned Scope	Implemented Scope
Multi-gym membership concept	Yes	Yes
Flexible walk-in access	Yes	Partially simulated through check-in page
Subscription billing	Yes	No
QR-based validation	Yes	No
Partner gym dashboard	Yes	No
User visit history	Yes	Yes
Tier-based access rules	Yes	Yes
Admin oversight	Yes	Basic version implemented

This table shows that the MVP remains faithful to the original problem framing while postponing infrastructure-heavy features.

4. Revenue Model and Business Viability

4.1 Revenue Model Concept

The original revenue model described in Progress Report 1 was a **subscription-based platform model**. Users would pay a monthly fee based on a tiered plan, and the platform would distribute a portion of value to participating gyms based on verified visits. This creates a hybrid model with at least three potential revenue layers:

- 1. **Base subscription revenue from end users**
- 2. **Commission retained by the platform from partner-gym payouts**
- 3. **Later-stage add-ons**, such as guest passes, premium features, advertising, or data services

This type of model is attractive because it creates recurring revenue while aligning incentives across users, facilities, and the platform operator.

4.2 Subscription Tier Concept from the Business Plan

The early business plan suggested several commercial pricing options, such as basic, standard, and premium tiers with differing monthly visit volumes and pricing. That market-facing pricing strategy is distinct from the simplified membership-tier logic in the MVP.

Original Business-Oriented Example Tiers

Tier	Proposed Monthly Price	Proposed Usage Model
Basic	\$49–\$69	8–12 visits/month
Standard	\$79–\$99	20–30 visits/month or semi-unlimited
Premium	\$129–\$159	Unlimited access

Implemented MVP Membership Tiers

Tier	Daily Visit Limit	Monthly Fee Used in App Logic
BRONZE	1	\$15.00
SILVER	2	\$30.00
GOLD	3	\$45.00

The implemented values are not a finalized commercial pricing model. Instead, they provide a lightweight way to encode access policy and demonstrate tier-based logic in software.

4.3 Effective Cost per Visit

The business case argued that a multi-gym subscription could provide users with strong value compared with paying for multiple separate gym memberships or repeated drop-in fees. The MVP supports this concept by displaying a **cost-per-visit estimate** on the home dashboard. This is calculated from the user's current monthly fee divided by total recorded visits.

This metric is useful for demonstration because it shows how platform logic can translate business policy into user-facing analytics. Although the current calculation is simple and does not reflect real billing cycles or partner payouts, it reinforces the product narrative: higher usage makes a subscription more efficient.

4.4 Gym Partner Revenue Logic in the Planned Model

In the planned platform, partner gyms would receive payouts based on actual verified visits. A commission retained by the platform would cover operations, support, infrastructure, fraud prevention, and growth. This is a common logic for platform businesses: the operator provides infrastructure and demand aggregation, and facilities receive monetized usage.

The final MVP does not implement payout calculations or end-of-month settlement. However, the system's check-in records and status values are foundational building blocks for such a future workflow.

4.5 ARR Projection and Growth Horizon

Progress Report 1 proposed a path to approximately **\$1,000,000 ARR within 3–4 years** through regional launch, network growth, and expansion into additional provinces. The exact numbers were based on assumptions regarding:

- active subscriber count;
- average revenue per user;
- partner-gym growth;
- churn;
- platform commission;
- later-stage add-ons.

Although these assumptions were not validated in the MVP, they remain useful as a strategic target because they define the long-term business ambition of the platform.

4.6 Revenue Projection Example

Phase	Illustrative Subscriber Range	ARPU Estimate	Strategic Outcome
Year 1	2,000–4,000	\$85/month	MVP launch and market validation
Year 2	8,000–12,000	\$90/month	Regional expansion
Year 3–4	12,000–15,000+	\$90/month	Approach or exceed \$1M ARR

These values came from the earlier business-planning stage and should be treated as exploratory strategic projections rather than validated operating results.

4.7 Financial Interpretation of the MVP

The final MVP does not process real payments, invoices, or payouts. As a result, it does **not** validate financial operations in the same way a production pilot would. However, it does validate several prerequisites for future monetization:

- membership tiers can be modeled;
- access can be restricted by plan logic;
- visits can be recorded and counted;
- account-level data can be persisted and displayed;
- administrative oversight can be layered on top of platform data.

These prerequisites are important because financial workflows depend on trustworthy event data. Without reliable check-in records and user membership state, billing and settlement would be impossible.

4.8 Commercial Risks

The business model also contains risk factors that must be acknowledged:

1. Partner acquisition may be slower than expected.
2. User value perception may vary depending on geographic coverage.
3. High-usage users could create margin pressure if pricing is not aligned to actual facility cost.
4. Fraud and account sharing must be controlled.
5. Payment disputes, tax handling, and data compliance would add operational complexity.

The MVP does not solve all of these, but it provides a controlled environment in which access rules and usage tracking can be explored.

5. Feasibility Study

5.1 Purpose of the Feasibility Study

Feasibility analysis was a major part of Progress Report 1. Its purpose was to determine whether the OneFitPass concept was realistic across market, technical, operational, financial, and scheduling dimensions. The final implemented product gives the team a stronger position from which to revisit these conclusions because the feasibility analysis can now be interpreted against an actual working system rather than a purely theoretical design.

5.2 Market Feasibility

The market analysis concluded that OneFitPass was feasible because the Canadian fitness market is large, fragmented, and currently underserved in the specific niche of **consumer-friendly, multi-location access to traditional gyms**. Existing alternatives validate demand for flexible access, but they do not fully solve the problem for independent gym users who want spontaneous or unscheduled visits.

The MVP supports this conclusion indirectly. By implementing a coherent user flow around multi-gym access, the software demonstrates that the concept can be represented in a tangible product. It does not prove product-market fit on its own, but it strengthens the plausibility of the business model.

5.3 Technical Feasibility

5.3.1 Feasibility in the Original Plan

The original plan proposed a modern web and mobile stack involving React, React Native, Node.js or NestJS, PostgreSQL, Redis, AWS, Stripe, and Auth0 or Cognito. This stack was assessed as technically feasible because it relies on mature and well-supported technologies.

5.3.2 Feasibility in the Implemented MVP

The final repository demonstrates technical feasibility through a different architecture: Java 17, Spring Boot, Thymeleaf, Spring Data JPA, and H2. This is also a mature technology stack, and in the context of an academic MVP it offers several advantages:

- rapid bootstrapping;
- lower infrastructure requirements;
- strong type safety;
- straightforward testing support;
- single-command local startup;
- well-understood MVC structure.

The change in stack does not invalidate the original feasibility conclusion. Instead, it shows that the platform can be implemented through more than one technically viable architecture.

5.4 Operational Feasibility

Operational feasibility refers to whether the system can be used and maintained effectively in its intended environment. In the MVP context, the application is operationally feasible because:

- it starts locally using the Maven Wrapper;
- it does not require an external database;
- demo users and demo gyms are automatically seeded;
- the interface is simple enough for demonstration and evaluation;
- test cases confirm expected functionality.

In a commercial production context, operational feasibility would require additional layers such as persistent hosting, backups, observability, payment support, incident handling, and gym partner support processes.

5.5 Economic Feasibility

The initial economic feasibility analysis suggested that the platform could become profitable if subscriber growth, partner participation, and unit economics behaved within expected ranges. It estimated manageable development and operational costs relative to long-term subscription revenue.

The MVP cannot validate commercial profitability directly, but it does reduce future uncertainty in two ways:

1. It proves that the core logic can be built with limited complexity.
2. It helps define which future features would be necessary to support monetization, such as billing, payout reconciliation, and reporting.

5.6 Schedule Feasibility

Schedule feasibility was strongly supported by the decision to reduce scope. The original concept was ambitious for an academic term because it included mobile workflows, partner systems, QR security, and billing. By narrowing delivery to a web-based MVP, the team produced a working system within course constraints.

This is an example of sound software engineering judgment: reducing scope does not mean failure; it often means choosing a realistic implementation strategy.

5.7 Legal and Compliance Feasibility

Progress Report 1 highlighted privacy, secure payment handling, and Canadian compliance considerations such as PIPEDA. The final MVP avoids the highest-risk compliance area—live payment handling—because payment integration was deferred. This makes the academic prototype safer and easier to manage. However, a real-world version would still need stronger privacy controls, security framework integration, consent handling, account recovery flows, and infrastructure-level compliance measures.

5.8 Feasibility Summary Table

Feasibility Type	Assessment	Rationale
Market	Feasible	Large fragmented fitness market with a recognizable access problem
Technical	Feasible	Both planned and implemented architectures rely on mature technologies
Operational	Feasible for MVP	Local startup, seeded data, and demonstrable workflows
Economic	Feasible in concept	Subscription-based model appears viable under stated assumptions
Schedule	Feasible after scope reduction	Delivered MVP fits academic time constraints
Compliance	Partially addressed	Basic account security present; production compliance still future work

5.9 Final Feasibility Conclusion

Taken together, the feasibility evidence remains positive. The project is feasible as a course-scale MVP and appears strategically feasible as the first step toward a broader product, provided that future iterations address persistent storage, payments, QR validation, operational monitoring, and stronger security controls.

6. Requirements Engineering and Analysis

6.1 Sources of Requirements

Requirements for OneFitPass came from multiple sources:

- the project problem statement and business goals;
- persona analysis;
- competitor gap analysis;
- feasibility considerations;
- expected course deliverables;
- final implementation choices made during development.

The final requirements picture includes both **planned requirements** from the earlier reports and **implemented requirements** confirmed in the repository.

6.2 Requirement Categories

The requirements can be organized into four main categories:

1. **Authentication and account management**
2. **Membership and gym access logic**
3. **Administrative visibility and system oversight**
4. **Non-functional quality attributes**, such as usability, maintainability, and testability

6.3 Functional Requirements from the Larger Vision

Progress Report 1 presented a broad functional scope including subscription purchase, payment processing, gym partner management, QR code entry, partner dashboards, and administrator controls. This section preserves those requirements for traceability while identifying final delivery status.

6.4 Functional Requirements Traceability Table

ID	Functional Requirement	Planned in Report 1	Implemented in Final Repository	Notes
FR-01	User registration with personal information	Yes	Yes (name, email, password, membership tier)	Simplified data set in MVP
FR-02	User login and logout	Yes	Yes	Session-based authentication
FR-03	Account role management	Yes	Partially	USER and ADMIN roles implemented
FR-04	Membership tier selection	Yes	Yes	Bronze, Silver, Gold
FR-05	Gym discovery / listing	Yes	Yes	Static seeded gym list
FR-06	Gym detail page	Yes	No	Not in MVP
FR-07	Gym check-in	Yes	Yes	Form-based check-in
FR-08	QR-based gym entry	Yes	No	Deferred
FR-09	Visit history	Yes	Yes	Reverse chronological list
FR-10	History filtering / search	Yes	No	Deferred
FR-11	Daily visit limit enforcement	Yes	Yes	Encoded by membership tier

ID	Functional Requirement	Planned in Report 1	Implemented in Final Repository	Notes
FR-12	Duplicate same-gym cooldown	Yes	Yes	30-minute rule
FR-13	Profile update	Yes	Yes	Name update implemented
FR-14	Password change	Yes	Yes	Current password verification implemented
FR-15	Membership change	Yes	Yes	Tier switch supported
FR-16	Payment processing	Yes	No	Deferred
FR-17	Billing history / receipts	Yes	No	Deferred
FR-18	Gym partner registration	Yes	No	Deferred
FR-19	Gym partner dashboard	Yes	No	Deferred
FR-20	Admin dashboard	Yes	Yes	Basic user and total check-in metrics
FR-21	Dispute resolution tooling	Yes	No	Deferred
FR-22	Exportable financial reports	Yes	No	Deferred

ID	Functional Requirement	Planned in Report 1	Implemented in Final Repository	Notes
FR-23	Seeded demo environment	Not central in Report 1	Yes	Added for demonstration convenience

6.5 Implemented Functional Scope Summary

The implemented system successfully covers the following essential user journeys:

1. A new user can register and immediately access the application.
2. An existing user can log in and receive a personalized dashboard.
3. A user can browse partner gyms and attempt a check-in.
4. The system enforces membership-based access rules.
5. The user can review historical check-ins.
6. The user can update profile data, change password, and change membership.
7. An administrator can review a basic system dashboard.

This set is appropriate for an MVP because it captures the core cycle of **register** → **subscribe conceptually** → **access gym** → **track usage** → **manage account**.

6.6 Non-Functional Requirements

The project also defined a broad set of non-functional requirements. Some were implemented directly, while others remain aspirational.

6.6.1 Usability

The application is server-rendered, form-driven, and visually consistent. Navigation is centralized via a shared Thymeleaf fragment, and the interface uses a responsive layout powered by a single CSS file. The check-in process is short and understandable, and the system provides success, warning, and error alerts.

6.6.2 Maintainability

Maintainability is one of the strongest implemented qualities. The repository is cleanly separated into controllers, services, repositories, models, configuration, templates, and static assets. This structure makes the code easier to understand, test, and extend.

6.6.3 Testability

The project includes unit tests, controller tests, and a context-loading test. The test suite passes successfully, confirming that the application is testable and that major pathways have automated verification.

6.6.4 Security

Passwords are hashed using BCrypt, and protected routes depend on session checks. This is appropriate for an academic prototype, although stronger framework-level security was deferred.

6.6.5 Portability

The application runs locally with the Maven Wrapper and an in-memory database. This significantly reduces environment friction and improves reproducibility for demonstrations and grading.

6.6.6 Performance and Scalability

These qualities were largely discussed in the design vision rather than fully validated in implementation. The current application is appropriate for local development and course demonstration, but it is not architected or tested for production-scale usage.

6.7 Non-Functional Requirement Mapping Table

NFR Category	Requirement Intent	MVP Status	Evidence
Usability	Simple, understandable user flow	Implemented	Consistent navigation, form-based actions, alerts
Maintainability	Clear modular structure	Implemented	Layered package design
Testability	Automated validation of critical behavior	Implemented	23 passing tests
Security	Protect credentials and routes	Partially implemented	BCrypt + session guards
Portability	Easy setup and demo readiness	Implemented	Maven Wrapper + H2
Scalability	Support large concurrent user counts	Not validated	No load testing or production infra
Reliability	Stable check-in and data recording	Partially implemented	Local persistence works, but DB resets on shutdown
Compatibility	Browser-based access	Implemented	Server-rendered HTML pages

6.8 Use Cases Covered by the MVP

The following use cases are fully or largely supported:

- Register account
- Log in
- View dashboard
- Check in at gym
- View history
- Update profile name

- Change password
- Change membership
- View admin dashboard

The following use cases remain out of scope for the final repository:

- Purchase subscription using online payment
- Scan QR code for access
- Register as partner gym
- Review payout statements
- Resolve disputes through workflow tooling
- Search or filter historical visits

6.9 Requirement Prioritization (MoSCoW View)

Priority	Requirements
Must Have	Registration, login, membership logic, gym listing, check-in, history, account management, admin view
Should Have	Search/filter history, richer admin analytics, stronger session/security framework
Could Have	QR check-in, gym details, notifications, role expansion
Won't Have in MVP	Full partner portal, payment processing, cloud deployment pipeline, mobile app

6.10 Requirements Conclusion

The requirements engineering phase was successful because it produced a large, ambitious platform vision while still allowing a realistic MVP to emerge. The final implementation does not satisfy the complete original requirement set, but it does satisfy the most critical requirements necessary to demonstrate the core idea. This is a positive outcome from a software engineering perspective because it shows prioritization, scope control, and traceability.

7. Design Goals and Scope Realignment

7.1 Role of Design Goals

In Progress Report 2, design goals were intended to translate non-functional requirements into concrete architectural decisions. This remains a useful lens for evaluating the final implementation. Even though the final architecture differs from the initially proposed stack, the same quality goals still apply: usability, maintainability, security, simplicity, and demonstrability.

7.2 Primary Design Goals

The implemented solution can be understood through the following design goals:

- 1. **Deliver a coherent minimum viable product** rather than an incomplete large-scale platform.
- 2. **Preserve clear separation of concerns** through layered architecture.
- 3. **Make local deployment and demonstration easy** using minimal infrastructure.
- 4. **Encode business rules centrally** to keep behavior consistent and testable.
- 5. **Use familiar and stable technologies** suitable for academic delivery and evaluation.

7.3 Design Goal Mapping Table

Design Goal	Related NFRs	Implemented Decision
Simplicity of deployment	Portability, operability	Maven Wrapper + H2 in-memory DB
Separation of concerns	Maintainability, testability	MVC layers: controller, service, repository, model
Fast demonstrability	Usability, schedule feasibility	Server-rendered UI, seeded data
Business rule consistency	Reliability, maintainability	Check-in logic centralized in service layer
Secure handling of credentials	Security	BCrypt password hashing
Basic access control	Security, usability	Session-based route protection
Easy extensibility	Maintainability	Modular packages and clean entity model

7.4 Why the Implemented Architecture Differs from the Planned Architecture

The earlier reports described a future-oriented platform based on web and mobile clients, cloud infrastructure, payment integration, and external authentication services. That design was valid from a product strategy perspective, but it was not the most practical implementation path for a course-scale deliverable.

The final architecture shifted to a Spring Boot MVC monolith for four major reasons:

1. **Reduced delivery risk** — a single deployable web application is easier to finish reliably within a short academic schedule.
2. **Lower integration overhead** — removing separate frontend/mobile stacks avoids time spent on API contracts, deployment coordination, and device-specific UI work.
3. **Faster demonstration** — a local web app with seeded data is easier for reviewers to run and inspect.

4. **Better traceability** — a simpler architecture makes it easier to connect code modules to business requirements in a final report.

This change should be interpreted as intentional scope management rather than design inconsistency.

7.5 Design Constraints in the MVP

The final design also reflects several constraints:

- no persistent production database;
- no mobile application;
- no external payment or identity provider;
- no Spring Security integration;
- no frontend JavaScript application;
- no cloud deployment topology.

These constraints kept the project manageable but also define the main boundaries of the MVP.

7.6 Design Philosophy of the Final Product

The final system reflects a pragmatic design philosophy: implement the **essential domain logic** first, then leave room for future infrastructure expansion. The project therefore prioritizes **clarity over complexity**, **demonstrability over completeness**, and **core workflow integrity over large-scale ecosystem simulation**.

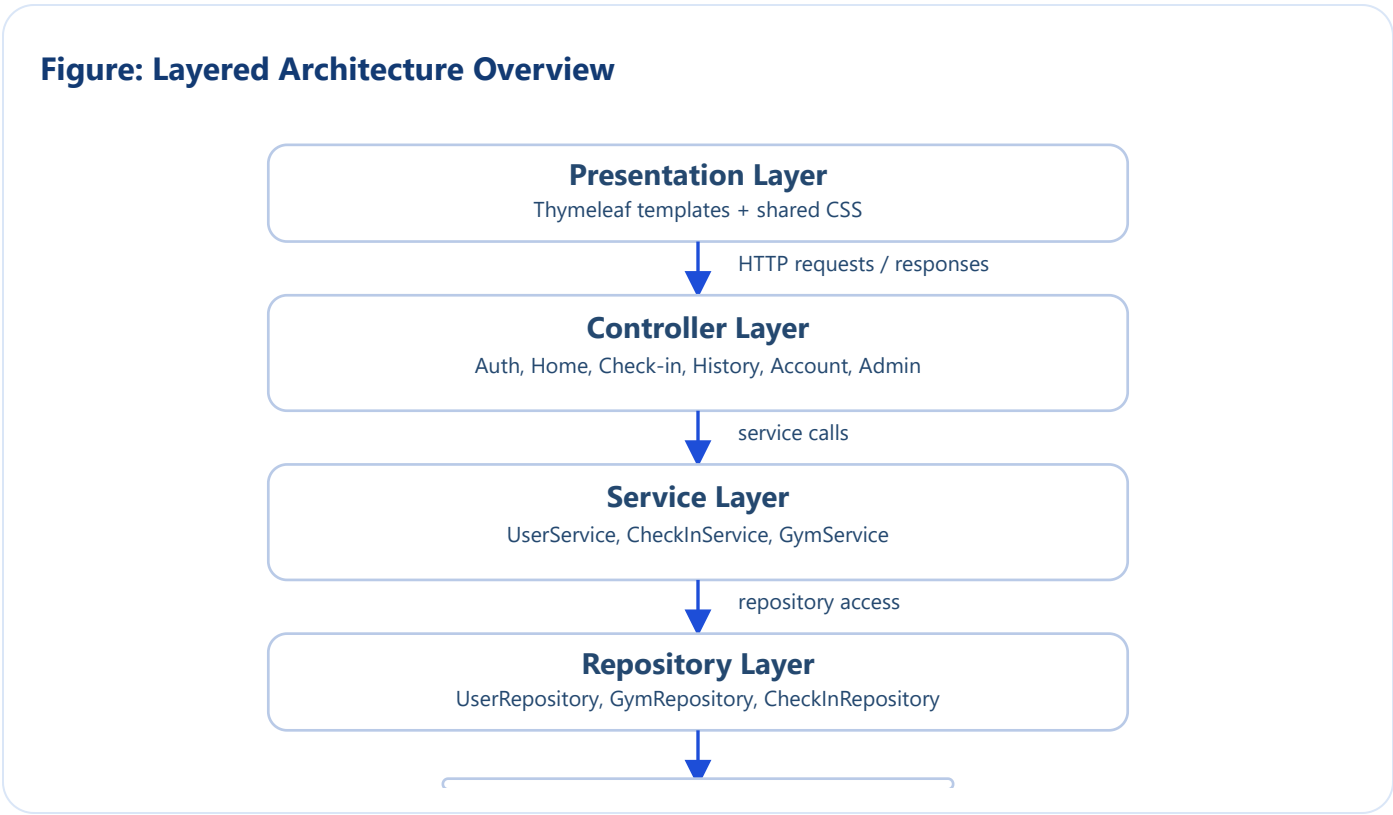
8. System Architecture and Technology Stack

8.1 Architectural Style

The final application uses a **monolithic layered MVC architecture**. This means that presentation, request handling, business logic, and persistence all exist within one deployable Spring Boot application, but each concern is separated into its own layer.

This architectural choice is appropriate for an academic MVP because it simplifies development, testing, and deployment while still preserving professional software structure.

8.2 Layered Architecture Overview



8.3 Technology Stack Table

Layer	Technology	Version / Detail	Purpose
Language	Java	17	Core implementation language
Framework	Spring Boot	3.4.4	Application bootstrapping and configuration
Web layer	Spring MVC	Included via starter-web	Request routing and MVC controller handling
Templating	Thymeleaf	Starter-based	Server-rendered user interface
Persistence	Spring Data JPA	Starter-based	Repository abstraction and ORM integration
ORM	Hibernate	Pulled through Spring Data JPA	Entity mapping
Database	H2	Runtime dependency	In-memory relational database
Security utility	jBCrypt	0.4	Password hashing and verification
Build tool	Maven Wrapper	<code>mvnw / mvnw.cmd</code>	Reproducible local build
Testing	JUnit 5, Mockito, MockMvc	starter-test	Unit, controller, and integration testing
UI styling	CSS	1 file, ~719 lines	Shared visual design across pages

8.4 Repository Structure

```
com.example.onepass
├── OnePass Application.java
├── config
│   └── DataLoader.java
├── controller
│   ├── AuthController.java
│   ├── HomeController.java
│   ├── CheckInController.java
│   ├── HistoryController.java
│   ├── AccountController.java
│   └── AdminController.java
├── model
│   ├── User.java
│   ├── Gym.java
│   ├── CheckIn.java
│   └── MembershipType.java
├── repository
│   ├── UserRepository.java
│   ├── GymRepository.java
│   └── CheckInRepository.java
└── service
    ├── UserService.java
    ├── CheckInService.java
    └── GymService.java
```

8.5 Architectural Narrative

When a user interacts with the application through the browser, the request first enters a **controller**. Controllers are intentionally thin: they validate session presence, receive form parameters, and delegate business operations to the relevant service. Services then implement domain logic, such as registering users, authenticating credentials, enforcing membership-based visit rules, or retrieving historical data. Repositories provide persistent access through Spring Data JPA. Finally, the result is passed into a Thymeleaf template and rendered as a complete HTML page.

This flow is consistent across all major routes and is one of the reasons the codebase remains easy to reason about.

8.6 Configuration Highlights

The `application.properties` file confirms the following runtime configuration:

- application name: `OnePass`
- datasource: `jdbc:h2:mem:onepass ;DB_CLOSE_DELAY=-1`
- H2 console enabled at `/h2-console`
- `spring.jpa.hibernate.ddl-auto=create-drop`
- `spring.jpa.show-sql=true`
- server port: `8080`

These settings make the system suitable for local development and demo use. They are not production-safe defaults, especially because the database is recreated on each run.

8.7 Seeded Data Architecture

The `DataLoader` class acts as a startup seed mechanism. On application launch, it inserts:

- eight gyms;
- one demo user account;
- one admin account.

This is a strong design choice for academic demonstration because the system is immediately usable after startup.

8.8 Architectural Strengths

The final architecture offers several strengths:

- easy for reviewers to run locally;
- clear separation between routing, logic, and persistence;
- good fit for server-rendered workflows;
- straightforward unit and controller testing;
- low environment complexity.

8.9 Architectural Limitations

The architecture also has known limitations:

- authentication is manually handled rather than managed by Spring Security;
- persistence is temporary because of H2 in-memory mode;
- no API layer exists for mobile or third-party consumers;
- no distributed scaling or production operations model is defined.

These limitations are acceptable for the academic MVP but would require future redesign decisions for production use.

9. Hardware and Software Configuration

9.1 Purpose of the Configuration Section

Progress Report 2 required documentation of the hardware and software environment necessary to run the system. Because the final application is a local web application, its environment is intentionally lightweight.

9.2 Software Requirements

Requirement	Detail
Operating system	Windows, macOS, or Linux capable of running Java 17
Java runtime	Java 17
Build tooling	Maven Wrapper included with the repository
Browser	Any modern web browser
Network	Internet connection required on first run for dependency download

9.3 Hardware Requirements

Hardware Aspect	Recommended Baseline
CPU	Modern dual-core or better
RAM	4 GB minimum, 8 GB recommended
Storage	Sufficient space for JDK, repository, and Maven cache
Display	Standard laptop or desktop display

Because the application uses an in-memory database and has no heavy frontend build pipeline, hardware requirements are modest.

9.4 Development Environment Characteristics

The project was reviewed in a Windows environment and started using `mvnw.cmd`. The use of the Maven Wrapper is important because it removes the need for a globally installed Maven version, improving reproducibility across machines.

9.5 Runtime Environment

At runtime, the application starts an embedded web server and listens on port **8080**. The H2 console is exposed at `/h2-console`. Because the database exists only in memory, no additional database process or service setup is required.

9.6 Production Readiness Consideration

Although the application is easy to run locally, the current hardware and software configuration is intentionally development-oriented. A production-grade version would require:

- persistent relational storage;
- secret management;
- environment variable configuration;
- HTTPS termination;
- backup strategy;
- health monitoring and logging;
- containerization or deployment automation.

The fact that these are absent is not a flaw in the academic context; rather, it is evidence that the delivered system is an MVP rather than a hosted service.

10. Database Design

10.1 Database Choice

The final implementation uses an **H2 in-memory relational database**. This choice supports the goals of simplicity, rapid demonstration, and zero external setup. It is particularly effective for a course project because it allows the entire application to run with minimal friction.

10.2 Core Entities

The database design centers on three persistent entities and one supporting enumeration:

- User
- Gym
- CheckIn
- MembershipType (enum stored as string)

10.3 Entity Descriptions

10.3.1 User

The `User` entity stores identity, login, membership, balance, creation date, and authorization role. It is the main account object in the system and acts as the parent entity for user check-ins.

10.3.2 Gym

The `Gym` entity stores the gym name and location. In the MVP, gyms are seeded and represent available partner locations.

10.3.3 CheckIn

The `CheckIn` entity links a user to a gym at a specific timestamp and stores a status value, such as `Successful`, `Too Soon`, or `Limit Reached`. This entity is central to platform analytics and future billing or reporting workflows.

10.3.4 MembershipType

The `MembershipType` enum captures business rules for each tier, including daily visit limit and monthly fee. This is an elegant domain design decision because it keeps plan logic close to the model.

10.4 Table Design

10.4.1 Table: users

Column	Type	Meaning / Constraint
id	BIGINT	Primary key, identity
name	VARCHAR	Not null
email	VARCHAR	Unique, not null
password	VARCHAR	Hashed password, not null
membership_type	ENUM / VARCHAR representation	Not null
balance	DOUBLE	Current balance field
created_at	DATE	Account creation date
role	VARCHAR	USER or ADMIN

10.4.2 Table: gyms

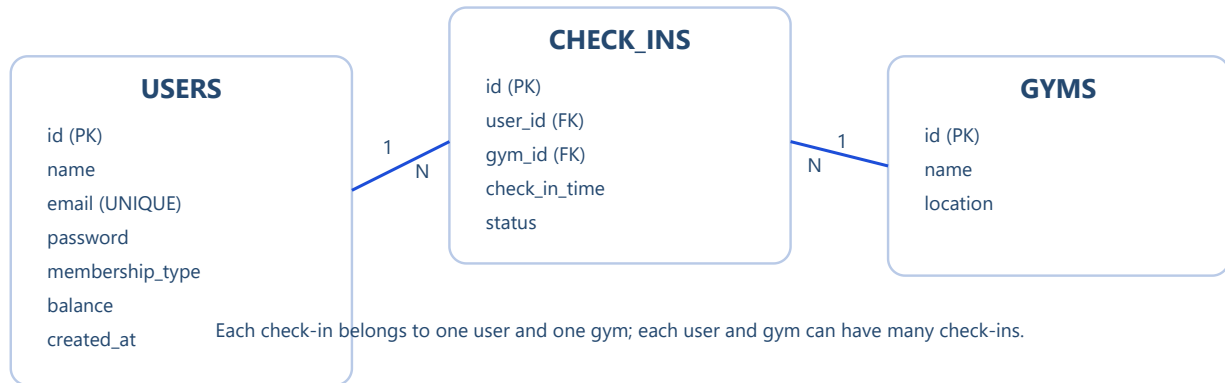
Column	Type	Meaning / Constraint
id	BIGINT	Primary key, identity
name	VARCHAR	Not null
location	VARCHAR	Not null

10.4.3 Table: check_ins

Column	Type	Meaning / Constraint
id	BIGINT	Primary key, identity
user_id	BIGINT	Foreign key to users.id
gym_id	BIGINT	Foreign key to gyms.id
check_in_time	TIMESTAMP	Not null
status	VARCHAR	Outcome of check-in

10.5 Entity-Relationship Diagram

Figure: Entity-Relationship Diagram



10.6 Normalization Discussion

Progress Report 2 specifically required normalization to at least third normal form for a relational database solution. The final schema satisfies this expectation:

First Normal Form (1NF)

All tables contain atomic values, and there are no repeating groups within a record.

Second Normal Form (2NF)

All non-key attributes depend on the full primary key of their table. Because each table uses a single-column surrogate key, partial dependency issues do not arise.

Third Normal Form (3NF)

Non-key fields depend directly on the primary key rather than transitively through another non-key field. For example:

- user attributes depend on `users.id` ;
- gym attributes depend on `gyms.id` ;

- check-in event attributes depend on `check_ins.id` ;
- membership type values are represented through an enum rather than duplicating plan metadata in separate event rows.

The design is therefore normalized enough for the project scope while remaining simple.

10.7 Persistence Behavior

The system uses `ddl-auto=create-drop` , which means the schema is created when the app starts and dropped when it stops. This is highly convenient for demos but means data is **not persistent across runs**. For a future production version, this would be replaced by a persistent database and a migration strategy.

10.8 Repository Methods and Query Behavior

The repository layer implements key access patterns:

- `findByEmail` and `existsByEmail` in `UserRepository`
- default CRUD operations in `GymRepository`
- ordered history retrieval, daily counting, total counting, and cooldown checks in `CheckInRepository`

These methods support the main domain questions the application needs to answer:

- who is the user?
- what plan do they have?
- what gyms exist?
- has this user checked in too recently?
- how many times has this user checked in today?
- what is the user's visit history?

10.9 Database Design Assessment

The database design is strong for an MVP. It is simple, understandable, normalized, and directly aligned to the implemented workflows. Its primary limitation is not schema quality but runtime

persistence and operational durability, both of which are intentionally outside the MVP scope.

11. Interface Design

11.1 Interface Design Philosophy

The final application uses a server-rendered HTML interface with shared styling and reusable navigation. The interface design prioritizes:

- low friction;
- clear navigation;
- minimal steps per task;
- consistency across pages;
- readability during demonstration.

This aligns well with the non-functional goals of usability and maintainability.

11.2 Presentation Architecture

The UI is composed of:

- **8 primary screens:** login, register, home, check-in, history, account, membership, and admin;
- **1 shared fragment:** navigation bar;
- **1 shared stylesheet:** `style.css` with approximately 719 lines.

No JavaScript files were found in the repository, which means the interface is almost entirely driven by HTML forms, server responses, and CSS styling.

11.3 Shared Navigation Design

The application uses a shared Thymeleaf fragment for the navigation bar. This improves consistency and reduces template duplication. The navigation provides links to Home, Check-in, History, Account, Membership, and Logout, with the Admin link appearing only when the session role is `ADMIN`.

This is a good design choice because role-aware navigation supports usability and basic authorization signaling.

11.4 Screen Inventory

Screen	URL	Primary Purpose
Login	/login	Authenticate an existing user
Register	/register	Create a new account
Home	/home	Show dashboard stats and quick actions
Check-in	/checkin	List gyms and submit a check-in
History	/history	Display past check-ins
Account	/account	Show account details and profile forms
Membership	/membership	Compare and change tiers
Admin	/admin	Show system and user summaries

11.5 Login and Registration Pages

The login and registration pages use a dedicated authentication layout with centered card design, prominent branding, and clean form fields. The login page also includes a demo credential hint, which is practical for demonstration. The registration form supports name, email, password, and membership-tier selection.

From a usability standpoint, the auth pages are clear and easy to use. From a design standpoint, the pages share a common visual identity through the blue gradient background and card-based layout.

11.6 Home Dashboard

The home page acts as the user’s dashboard and includes:

- membership status card;
- today's visits count;
- total check-ins;
- cost-per-visit metric;
- balance display;
- quick action buttons.

This is a strong dashboard design for an MVP because it combines account state with activity state and provides direct next steps.

11.7 Check-in Page

The check-in page presents gyms in a card-based grid layout. Each card displays the gym name, location, and a one-click check-in button. Alerts are used to communicate success, warning, or error states.

The absence of search or filtering means scalability is limited, but for the current seeded data set of eight gyms, the design is practical and understandable.

11.8 History Page

The history page provides a reverse-chronological list of check-ins and shows a total count. It also includes an empty state with a direct call to action when no visits exist. Successful and warning statuses are visually differentiated, which helps communicate business rule outcomes.

11.9 Account Page

The account page is more than a simple profile view. It combines:

- account metrics;
- personal information;
- name update form;
- password change form;
- links to membership management and logout.

This page is functionally dense but well-structured through card-like sections.

11.10 Membership Page

The membership page presents all tiers in a comparison grid and visually highlights the current plan. This is a helpful design because it exposes plan alternatives in a way that is immediately understandable and supports plan change in a single form submission.

11.11 Admin Page

The admin page provides system-level stats and a user table showing ID, name, email, membership, role, balance, and member-since date. This is a limited but appropriate administrative interface for an MVP.

11.12 Styling and Responsiveness

The shared stylesheet includes dedicated rules for:

- auth pages;
- forms;
- buttons;
- alerts;
- navigation;
- page containers;
- stats grid;
- gym grid;
- history list;
- account cards;
- tier cards;
- admin table.

The CSS is responsive in the sense that many sections use flexible layouts and CSS grid, such as `stats-grid` , `gym-grid` , and `tier-grid` . This supports different screen widths without requiring separate mobile code.

11.13 UI Design Strengths

The interface has several notable strengths:

- strong consistency due to shared fragment and shared CSS;
- clear information hierarchy;
- limited cognitive load per screen;
- meaningful status feedback;
- professional visual style for a course project.

11.14 UI Design Limitations

The current interface also has limitations:

- no search/filtering in gym list or history;
- no JavaScript-enhanced interactions;
- no QR or camera workflow;
- no multi-language support;
- no accessibility audit documented.

These limitations are acceptable given the MVP scope but should be addressed in later iterations.

12. UML and System Diagrams

12.1 Purpose of the Diagram Section

Progress Report 2 required revised UML diagrams reflecting the solution domain. The final implementation supports a reliable set of class, use case, sequence, activity, and component representations. The diagrams below are presented in textual and code-block form so they can be recreated in diagramming tools if needed.

12.2 Use Case Overview

Main Actors

- Regular User
- Administrator

Supported User Use Cases

- Register account
- Log in / log out
- View dashboard
- Check in at a gym
- View check-in history
- Update profile
- Change password
- Change membership

Supported Admin Use Cases

- Log in
- View system dashboard

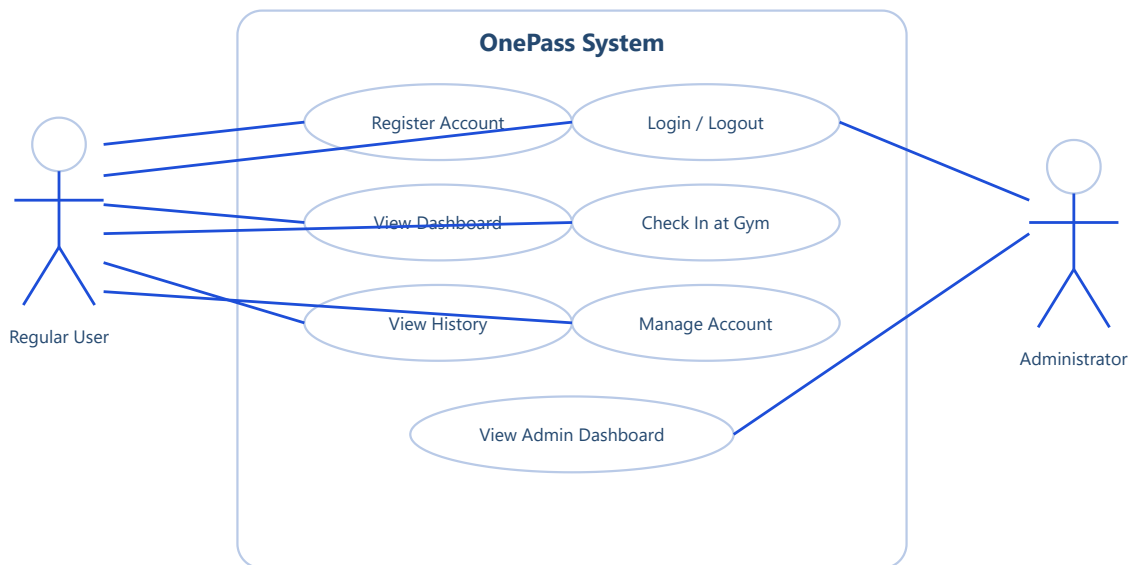
- Review user list and total check-ins

Deferred Use Cases

- Purchase subscription
- Scan QR code
- Register partner gym
- Review payouts
- Resolve disputes

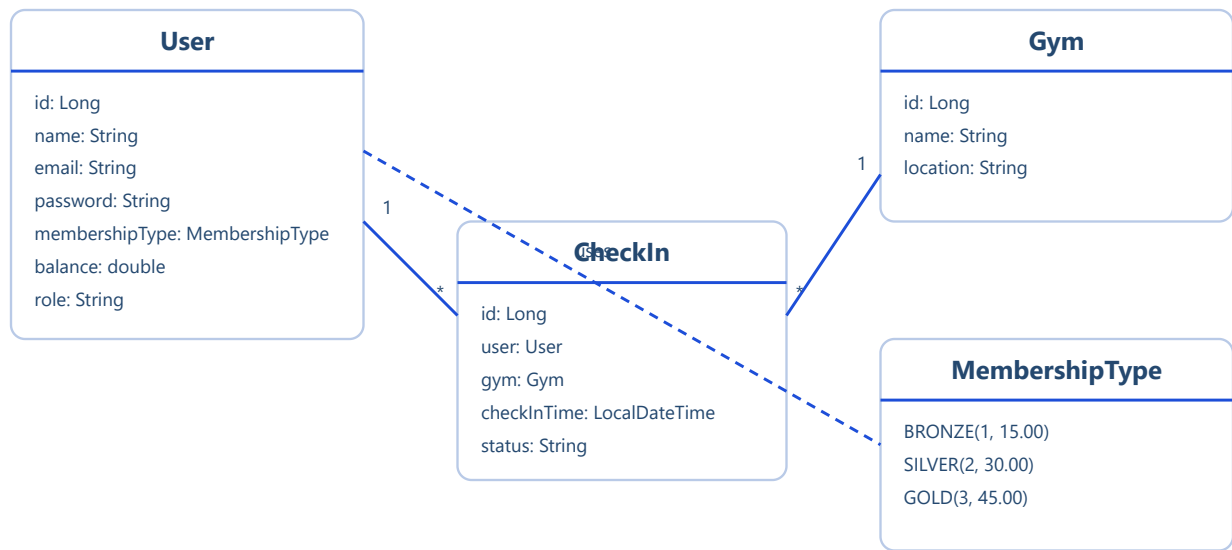
12.3 Textual Use Case Diagram

Figure: Use Case Diagram



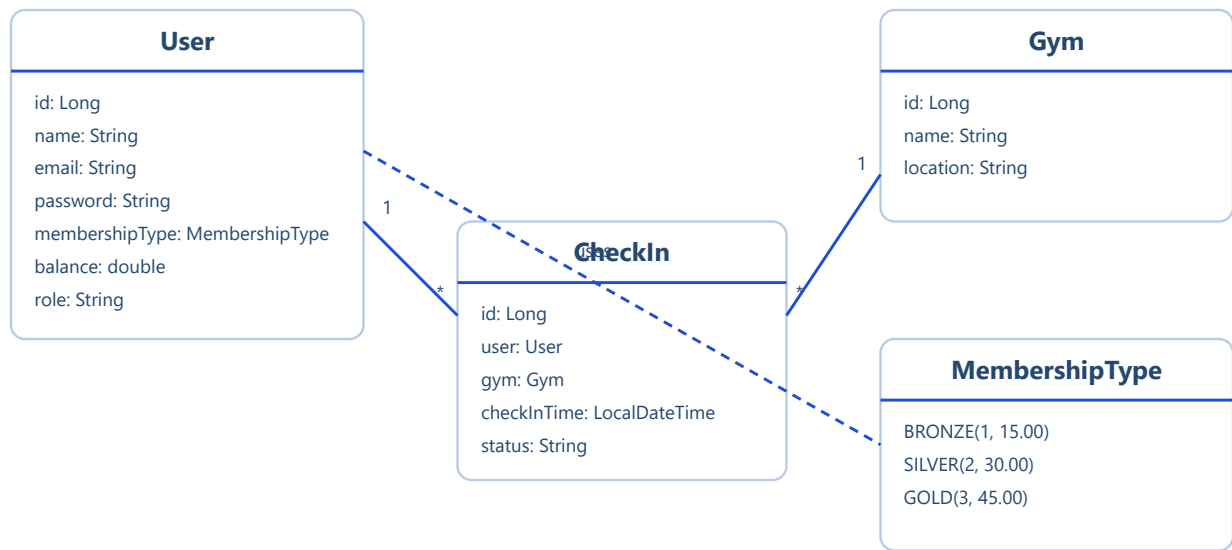
12.4 Class Diagram

Figure: Core Class Diagram



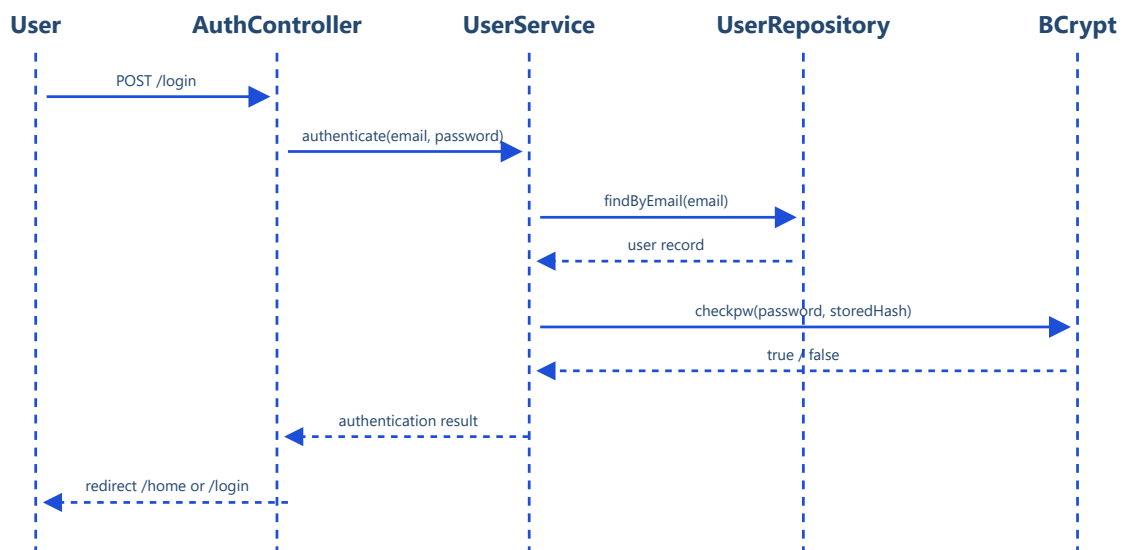
12.5 Mermaid Version of the Class Diagram

Figure: Core Class Diagram



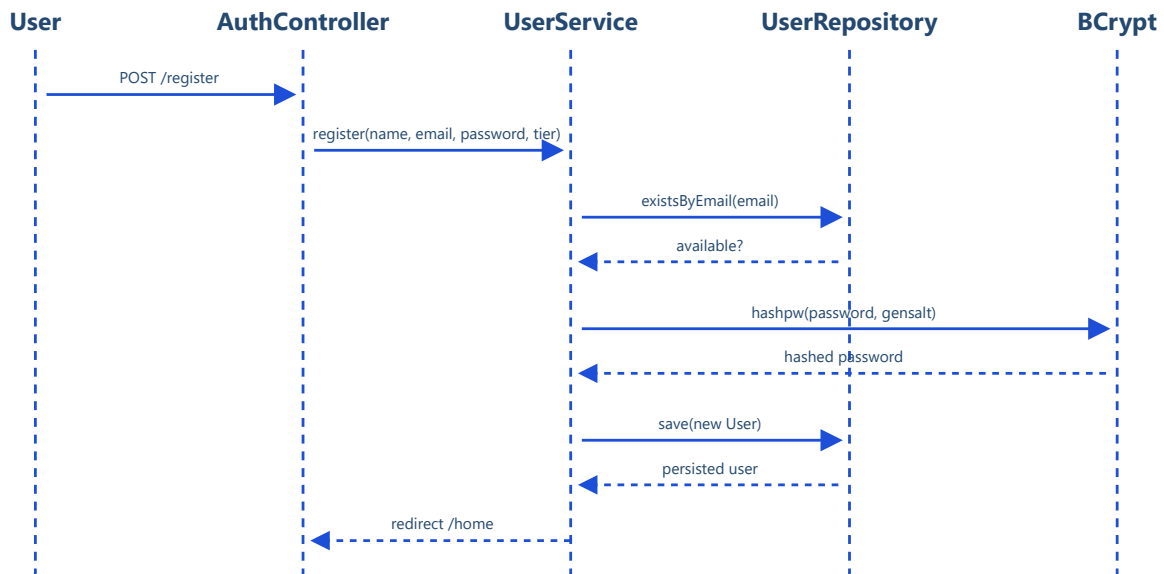
12.6 Sequence Diagram — Authentication Flow

Figure: Authentication Sequence Diagram



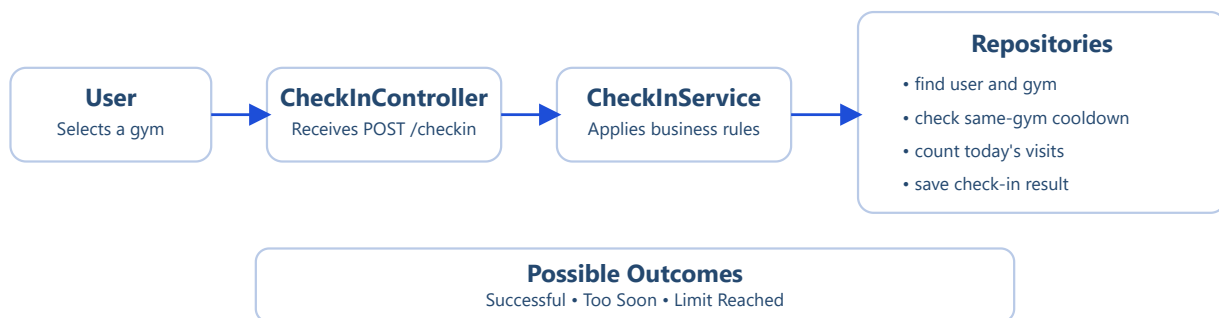
12.7 Sequence Diagram — Registration Flow

Figure: Registration Sequence Diagram



12.8 Sequence Diagram — Check-In Flow

Figure: Check-In Flow Diagram

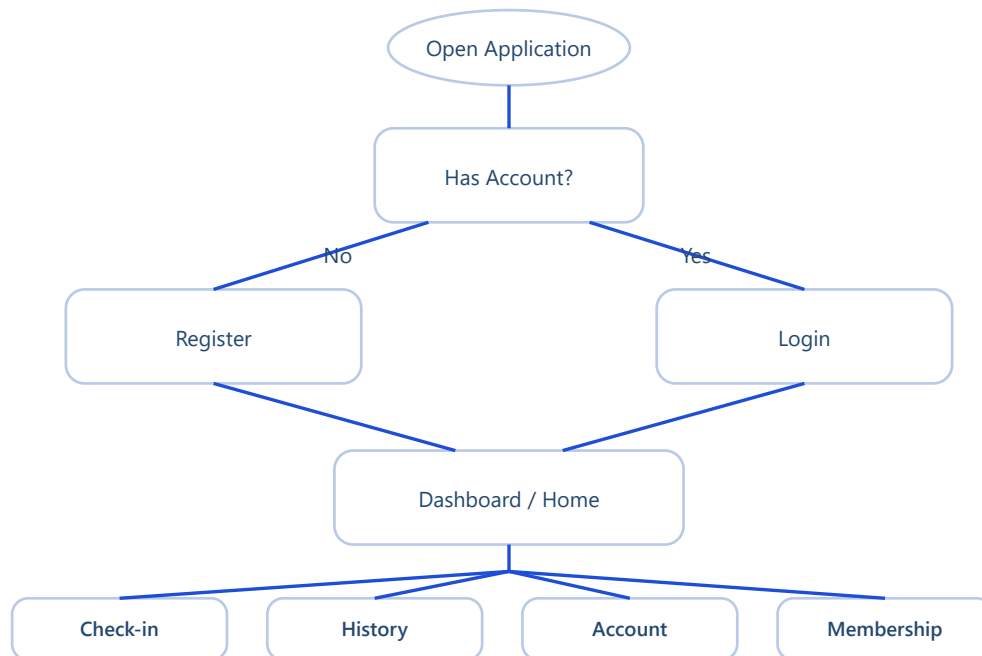


12.9 Sequence Diagram — Membership Change

```
User -> AccountController: POST /membership/change
AccountController -> UserService: changeMembership(userId, newTier)
UserService -> UserRepository: findById(userId)
UserRepository --> UserService: user
UserService: update membershipType and balance
UserService -> UserRepository: save(user)
UserRepository --> UserService: saved user
UserService --> AccountController: success
AccountController --> User: redirect /membership
```

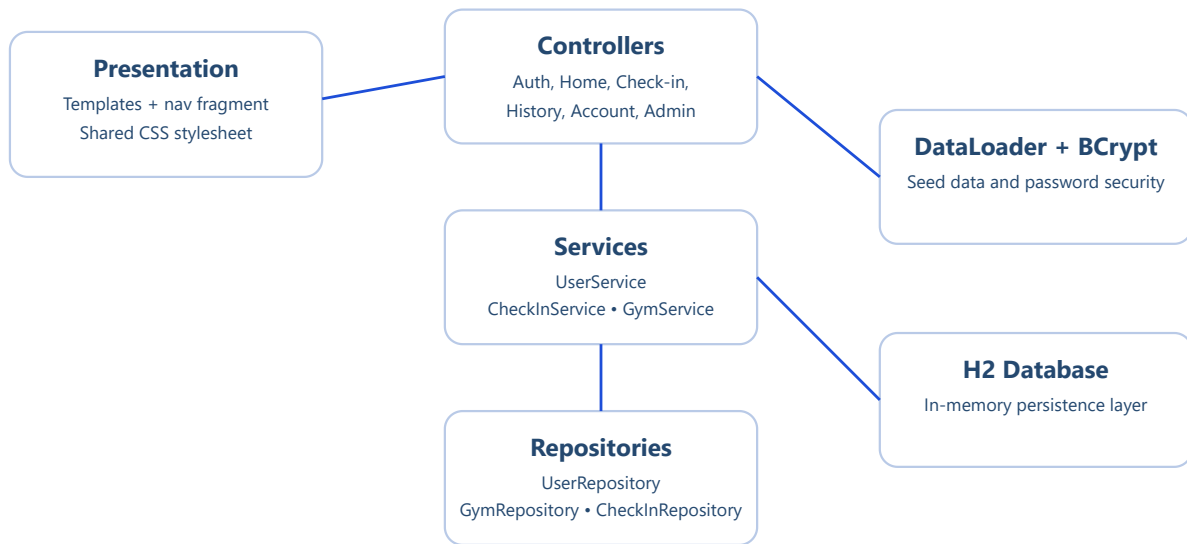
12.10 Activity Diagram — User Journey

Figure: User Journey Activity Diagram



12.11 Component Diagram

Figure: Component Diagram



Presentation Components:

- login.html
- register.html
- home.html
- checkin.html
- history.html
- account.html
- membership.html
- admin.html
- fragments/nav.html
- static/css/style.css

Application Components:

- Controllers
- Services
- Repositories
- DataLoader
- BCrypt utility

Data Component:

- H2 database

12.12 Diagram Interpretation

The diagrams confirm that the final solution is centered on a small but well-defined domain model and a predictable set of user interactions. The most important engineering pattern is that business rules are implemented in the service layer rather than scattered across controllers or templates. This improves maintainability and makes the logic easier to test.

13. Final Implemented Software Product

13.1 Delivered Scope Overview

The final implemented software product is a working Spring Boot web application that realizes the core functions of the OnePass MVP. The delivered scope can be summarized as follows:

1. Register a new user account with a selected membership tier.
2. Log in and log out through session-based authentication.
3. View a personalized home dashboard.
4. Browse a list of available gyms.
5. Check in at a selected gym.
6. Enforce membership-tier visit rules.
7. Record and display visit history.
8. Update profile information and password.
9. Change membership tier.
10. Access an administrator dashboard when authorized.

13.2 Application Entry Point

The application starts from `OnePass Application.java`, which boots the Spring Boot context. From there, the embedded web server exposes the configured routes and loads the H2-backed persistence layer.

13.3 Controller Layer Summary

Controller	Routes / Purpose
AuthController	Root redirect, login, register, logout
HomeController	Home dashboard
CheckInController	Check-in page and check-in submission
HistoryController	User history page
AccountController	Account page, profile update, password change, membership page and membership change
AdminController	Admin dashboard

Controllers are intentionally thin and mostly handle session checks, request parameters, flash messages, and view selection.

13.4 Service Layer Summary

Service	Responsibility
UserService	Registration, authentication, profile update, password change, membership change, user lookups
CheckInService	Check-in rule evaluation, history retrieval, visit counting
GymService	Gym retrieval

The service layer is the main carrier of business logic and is one of the cleanest aspects of the implementation.

13.5 Repository Layer Summary

Repository	Responsibility
UserRepository	User lookup by email, existence checks, CRUD
GymRepository	Gym CRUD and lookup
CheckInRepository	History ordering, count queries, cooldown query

13.6 Core Domain Logic

13.6.1 Registration

User registration validates:

- non-blank name;
- valid email pattern;
- minimum password length of 6 characters;
- email uniqueness.

If validation passes, the password is hashed using BCrypt and the user is stored with an initial balance equal to the selected membership type's monthly fee.

13.6.2 Authentication

Authentication checks the submitted email and password against stored account data. If successful, the application stores `userId`, `userName`, and `userRole` in the `HttpSession`. If authentication fails, the user is redirected back to the login page with an error message.

13.6.3 Check-In Business Rules

Check-in behavior is controlled by two business rules:

1. **30-minute same-gym cooldown** — a user cannot repeatedly check in to the same gym within 30 minutes without receiving a warning outcome.
2. **Daily tier limit** — Bronze allows 1 visit/day, Silver 2 visits/day, Gold 3 visits/day.

The system saves a `CheckIn` record regardless of whether the result is `Successful` , `Too Soon` , or `Limit Reached` . This design is useful because it preserves the event trail.

13.6.4 History Retrieval

The history page retrieves a user’s check-ins ordered by descending timestamp. This lets the user review prior activity and see the status of each event.

13.6.5 Account Management

Users can update their name, change password with current-password verification, and switch membership tiers. Membership changes also update the balance field to match the new tier fee.

13.7 Membership Tier Design

Tier	Daily Visit Limit	Monthly Fee
BRONZE	1	\$15.00
SILVER	2	\$30.00
GOLD	3	\$45.00

These values are implemented in the `MembershipType` enum and therefore remain consistent throughout the application.

13.8 Seeded Demo Data

13.8.1 Seeded Gyms

1. Gold's Gym Downtown
2. Fitness First Midtown
3. Planet Fitness Central
4. Anytime Fitness
5. LA Fitness Eastside
6. Equinox Downtown

7. Crunch Fitness North

8. 24 Hour Fitness South

13.8.2 Seeded Accounts

Account	Email	Password	Role
Demo User	alex@example.com	password123	USER
Admin	admin@onepass .com	admin123	ADMIN

13.9 Codebase Summary Metrics

Area	Count
Main Java source files	18
Controllers	6
Services	3
Repositories	3
Domain model files	4
HTML templates	9
Shared fragments	1 of those 9
CSS files	1
CSS lines	~719
Test classes	5

13.10 Strength of the Delivered Product

The implemented MVP is a strong academic deliverable because it is not merely a mockup or design-only project. It is a working application with business rules, persistent runtime data, reusable templates, automated tests, and clear deployment instructions.

13.11 Deferred Scope

The following features were intentionally not implemented in the final repository:

- history filtering and gym search on history page;
- QR-based check-in;
- payment processing and subscription billing;
- partner-gym self-service workflows;
- production database support;
- cloud deployment pipeline;
- mobile application support;
- advanced analytics and financial reporting.

13.12 Interpretation of MVP Completeness

The MVP is complete in the sense that it provides a full end-to-end user experience around the most important domain activities. It is incomplete only relative to the long-term product roadmap, which is normal and expected for a first iteration.

14. Security Design and Implementation

14.1 Security Scope in the MVP

Security in the final project is intentionally lightweight but meaningful. The system protects the most sensitive aspect of a basic web application—user credentials—and limits access to authenticated or authorized pages using server-side checks.

14.2 Authentication Mechanism

Authentication is implemented manually using `HttpSession` rather than through Spring Security. When a user successfully logs in, the application stores:

- `userId`
- `userName`
- `userRole`

Controllers then check whether a session exists before rendering protected pages. This design is simple and understandable, although it does not provide the robustness or default protections of a dedicated security framework.

14.3 Authorization

Authorization is role-based at a basic level. The admin dashboard checks whether `userRole` is equal to `ADMIN`. If not, the user is redirected away from the admin page.

This is sufficient for the MVP because there are only two roles and one admin-only route. A larger system would require more structured policy management.

14.4 Password Security

The most important implemented security measure is the use of **BCrypt**. Passwords are never stored in plain text. During registration and seed loading, passwords are hashed with a salt using

BCrypt. During authentication and password change operations, password verification uses `BCrypt.checkpw(...)` .

This is a strong choice for a course project because it demonstrates awareness of proper password storage practices.

14.5 Input Validation

The application implements validation at multiple levels:

- HTML form requirements (`required` , email input type, etc.);
- service-layer checks for blank names;
- email format regex validation;
- minimum password length validation;
- duplicate email prevention;
- current password verification for password change.

Validation in the service layer is especially important because it ensures that business rules remain enforced even if a request bypasses simple client-side checks.

14.6 Security-Relevant Routes

Route	Protection Strategy
/home	Requires session userId
/checkin	Requires session userId
/history	Requires session userId
/account	Requires session userId
/membership	Requires session userId
/admin	Requires session userId + ADMIN role

14.7 Session Handling

Logout invalidates the session using `session.invalidate()`. This is straightforward and effective for the application scope.

However, the current system does not implement:

- session timeout configuration;
- CSRF protection management;
- remember-me behavior;
- account lockout on repeated failure;
- device/session tracking.

These are all future hardening opportunities.

14.8 Security Strengths

1. Passwords are hashed securely.
2. Routes are protected at the server side.
3. Admin access is role-aware.
4. Sensitive changes, such as password updates, require credential confirmation.

14.9 Security Limitations

1. Manual session-based security is more error-prone than a dedicated framework.
2. No password reset or recovery flow exists.
3. No rate limiting or failed-login tracking is implemented.
4. No audit log exists for security-sensitive events.
5. Payment security is not addressed because payments are deferred.

14.10 Future Security Hardening Roadmap

Future iterations should consider:

- Spring Security integration;

- CSRF token handling and form hardening;
- password reset flow;
- login throttling;
- audit logging;
- optional MFA for administrators and partner roles;
- stronger separation between session handling and controller logic.

14.11 Security Assessment

Overall, security is appropriate for an academic MVP. The project correctly prioritizes hashed passwords and protected routes, while openly acknowledging that production-grade security would require a stronger framework and broader operational controls.

15. Testing Design and Testing Results

15.1 Testing Objectives

Testing in the final project serves three purposes:

- 1. verify that key business logic behaves correctly;
- 2. verify that controllers route requests as expected;
- 3. verify that the Spring Boot application context starts successfully.

The final repository includes a meaningful automated test suite that supports all three goals.

15.2 Testing Techniques Used

Technique	Purpose
Unit testing	Validate isolated service logic
Controller testing	Validate route behavior, views, and redirects
Integration / context testing	Validate application startup and wiring

15.3 Test Classes

Test Class	Type	Main Focus
UserServiceTest	Unit	Registration validation, duplicate prevention, authentication
CheckInServiceTest	Unit	Success, cooldown, daily limit, history, count logic
CheckInControllerTest	Controller	Session protection, page rendering, redirect behavior
HomeControllerTest	Controller	Session validation, dashboard rendering, stale user handling
OnePass ApplicationTests	Integration	Spring context loading

15.4 Verified Test Execution

The repository was tested using the command:

```
mvnw.cmd test
```

The observed results were:

Metric	Result
Total tests run	23
Failures	0
Errors	0
Skipped	0
Build result	BUILD SUCCESS
Total execution time	~14.032 s

15.5 Per-Class Results

Test Class	Tests	Result
CheckInControllerTest	4	Passed
CheckInServiceTest	5	Passed
HomeControllerTest	3	Passed
OnePass ApplicationTests	1	Passed
UserServiceTest	10	Passed
Total	23	All Passed

15.6 What the Tests Prove

The test suite establishes the following confidence points:

- registration input validation works as expected;
- duplicate email handling works;
- password verification logic works;
- check-in status logic works for success, cooldown, and daily limit cases;
- controllers correctly redirect unauthenticated users;
- the dashboard loads with valid session context;
- the full Spring context starts without failure.

15.7 Warnings Observed During Test Execution

The test run also reported several warnings that do not fail the build but are relevant for maintenance:

1. `@MockBean` usage is deprecated and marked for removal in future Spring Boot versions.
2. Explicit `H2Dialect` configuration is unnecessary because Hibernate can infer it automatically.
3. Spring Boot reported the default `spring.jpa.open-in-view` warning.

These warnings are helpful because they identify future cleanup opportunities.

15.8 Testing Gaps

Although the suite is solid for an MVP, several areas remain untested or under-tested:

- AuthController-specific integration of login and registration flows;
- AccountController and membership routes;
- AdminController authorization behavior;
- template rendering details beyond basic controller validation;
- seed data assertions;
- end-to-end browser interaction testing.

15.9 Quality Assessment of the Test Strategy

For a project of this size, the testing strategy is appropriate and above the minimum expected for a course deliverable. The combination of service, controller, and context tests demonstrates sound engineering practice and gives evaluators confidence that the application behavior is not purely anecdotal.

15.10 Suggested Future Testing Enhancements

Future testing could include:

- fuller controller coverage;
 - repository integration tests;
 - end-to-end smoke tests using browser automation;
 - property-based testing of check-in edge cases;
 - load testing once a production-oriented architecture exists.
-

16. Deployment, Operations, and Usage

16.1 Deployment Philosophy

The final project is designed for **local demonstration deployment** rather than production hosting. This is intentional and appropriate for an academic submission. The goals are fast startup, minimal setup, and predictable behavior.

16.2 Prerequisites

Requirement	Detail
Java	Version 17
Internet	Needed on first run for dependency download
Browser	Any modern browser

16.3 Startup Steps

1. Open a terminal in the project root.
2. Run the Maven Wrapper command.
3. Wait for the Spring Boot application to start.
4. Open the browser to `http://localhost:8080` .

Windows Command

```
mvnw.cmd spring-boot:run
```

macOS / Linux Command

```
./mvnw spring-boot:run
```

16.4 Demo Accounts

Role	Email	Password
Demo User	alex@example.com	password123
Admin	admin@onepass .com	admin123

16.5 H2 Console Access

The application exposes the H2 console at:

```
http://localhost:8080/h2-console
```

Use the following values:

Setting	Value
JDBC URL	jdbc:h2:mem:onepass
Username	sa
Password	blank

16.6 Operational Notes

- The database resets every time the application stops.
- Seed data loads automatically on startup.
- No external DB setup is required.
- The system is optimized for local demonstration, not persistent real-world operation.

16.7 Demonstration Script Suggestion

For presentation or grading, the following demo flow is effective:

1. Start the application.
2. Log in as `alex@example.com`.
3. Show the dashboard and membership card.
4. Navigate to Check-in and select a gym.
5. Show history update after check-in.
6. Demonstrate account update or membership change.
7. Log out and log in as admin.
8. Show the admin dashboard and total users/check-ins.

This walkthrough covers the main system value in a short period of time.

16.8 Operations Gaps for Production Use

The MVP does not yet provide:

- persistent hosting;
- backup strategy;
- structured environment management;
- observability stack;
- HTTPS termination;
- database migration tooling;
- customer support and incident procedures.

These are normal next steps once an MVP evolves toward a real deployment target.

17. KPI Evaluation

17.1 KPI Context

KPI evaluation in this project spans three phases:

- 1. analysis-phase KPIs from requirements work;
- 2. design-phase KPIs from architecture and solution modeling;
- 3. implementation-phase KPIs based on the final delivered product.

17.2 Requirement Analysis Phase KPIs

Progress Report 1 proposed KPIs related to requirement completeness, clarity, business alignment, technical feasibility, usability, financial viability, security coverage, and stakeholder validation.

Assessment

KPI Area	Assessment
Requirement completeness	Strong — broad feature inventory documented
Requirement clarity	Moderate to strong — most core requirements identifiable
Business alignment	Strong — requirements linked to revenue model and personas
Technical feasibility validation	Strong — detailed stack discussion provided
Usability orientation	Strong — core flows explicitly discussed
Financial viability link	Moderate — strategic rather than validated
Security coverage	Moderate — comprehensive at planning level

The strongest outcome of the analysis phase was that it created a rich foundation for the project, even though not every planned feature was later implemented.

17.3 Design Phase KPIs

Progress Report 2 focused on architecture, database design, interfaces, and solution diagrams. In the final repository, these can be evaluated against actual delivery.

Design KPI	Target	Final Assessment
Architecture defined	Clear system structure	Met
Database normalized	3NF relational design	Met
UI design represented	Distinct pages and flows	Met
Revised UML available	Class/sequence/activity representation	Met in final documentation
NFR-to-design mapping	Explicit design rationale	Met
Hardware/software configuration documented	Clear runtime requirements	Met

17.4 Implementation KPIs

The following table evaluates the final application against concrete implementation outcomes.

KPI	Target	Actual Result	Status
Account registration	New users can create accounts	Implemented	Met
Authentication	Login and logout works	Implemented	Met
Membership enforcement	Tier-based limits apply	Implemented	Met
Duplicate prevention	Same-gym cooldown applied	30-minute rule implemented	Met
Gym access flow	User can check in to a listed gym	Implemented	Met
Visit history	User can review prior check-ins	Implemented	Met
Account management	Name and password can be updated	Implemented	Met
Membership change	User can switch tiers	Implemented	Met
Admin oversight	Admin sees system/user summary	Implemented	Met
Demo readiness	Seeded gyms and accounts available	8 gyms, 2 accounts	Met
Test automation	Automated tests pass	23/23 passed	Met
UI coverage	Multiple connected pages exist	8 pages + nav fragment	Met
History filtering	Search/filter historical records	Not implemented	Deferred
Payment processing	Billing workflow exists	Not implemented	Deferred
QR check-in	QR or scanner experience exists	Not implemented	Deferred

KPI	Target	Actual Result	Status
Cloud deployment	Hosted deployment configuration exists	Not implemented	Deferred

17.5 KPI Summary

Based on the table above, **12 of 16 implementation KPIs are met** and **4 remain deferred**. This is a strong result for a course MVP because the implemented KPIs include nearly all core user-facing flows and platform rule enforcement.

17.6 KPI Interpretation

The KPI picture tells an important story: the project succeeded in delivering a working and testable core product, while deliberately postponing infrastructure-heavy or commercially advanced features. This is exactly the kind of trade-off that mature software projects often make in early release stages.

18. Risk Management, Limitations, and Future Roadmap

18.1 Risk Register Overview

The table below summarizes key risks relevant to both the project and any future expansion.

Risk	Description	Current Mitigation	Residual Concern
Scope creep	Too many features for one term	MVP scope reduction	Future phases still broad
Security under-hardening	Manual auth may miss protections	BCrypt + route guards	Needs Spring Security later
Data loss	In-memory DB resets on shutdown	Seed data supports demos	No persistence for production
Business-model uncertainty	Revenue assumptions not market-tested	Clear conceptual model	Requires real pilot validation
Partner dependency	Coverage depends on gym participation	Not applicable in MVP	Major commercialization challenge
Fraud / misuse	Shared accounts or invalid check-ins	Cooldown + tier limits	QR and location checks deferred
Technical debt	Fast MVP choices may constrain growth	Layered structure reduces some debt	Needs refactoring for production features

18.2 Current Limitations of the Repository

The final repository has several explicit limitations:

1. Data is temporary because H2 is in-memory.
2. Authentication is manual rather than framework-managed.
3. The interface does not support searching or advanced filtering.
4. Payment and billing workflows are absent.
5. Partner gym management is absent.
6. The application is not deployed to the cloud.
7. There is no mobile app.
8. There is no QR-based or hardware-assisted check-in flow.

These limitations are important to state because they clarify what the MVP is and is not intended to prove.

18.3 Planned vs Implemented Scope Reconciliation

One of the most important reflective tasks in this final report is reconciling the earlier ambitious plans with the delivered system. The most professional interpretation is as follows:

- The **problem statement** remained valid throughout the project.
- The **design direction** remained strategically valid.
- The **implementation scope** was narrowed to fit schedule, complexity, and demonstrability constraints.
- The **MVP preserves the domain model and major user journey**, making it a credible first iteration rather than a disconnected simplification.

18.4 Roadmap for Version 2

Short-Term Enhancements

- Add history search and date filtering.
- Add gym search and sort options.
- Expand controller test coverage.
- Add clearer validation feedback in more forms.
- Replace manual role checks with centralized authorization helpers.

Medium-Term Enhancements

- Introduce Spring Security.
- Persist data using PostgreSQL or another production-grade database.
- Add partner-gym entity model and admin management of gym records.
- Add richer dashboard metrics for admins.
- Add account recovery and reset flows.

Long-Term Enhancements

- Integrate payment provider for real subscriptions.
- Implement QR-based or token-based secure check-in.
- Add a mobile client or responsive PWA experience.
- Add cloud deployment and CI/CD automation.
- Build partner portal, payout logic, and financial reporting.

18.5 Production Transition Considerations

Transitioning from MVP to production would require changes across multiple layers:

- **Architecture:** possibly introduce APIs or service boundaries.
- **Persistence:** move to durable relational storage.
- **Security:** add framework-level authentication and authorization.
- **Operations:** add monitoring, logging, backups, and secret management.
- **Business workflows:** add billing, invoicing, tax handling, partner payout, and dispute resolution.

The good news is that the current MVP creates a clean enough foundation that many of these changes can be layered onto the existing domain model.

19. Lessons Learned and Reflection

19.1 Importance of Scope Control

One of the clearest lessons from this project is that a good software engineering outcome often depends more on **scope control** than on ambition alone. The original product vision was attractive and strategically sound, but too large to implement fully within a single academic term. By focusing on the core user journey and deferring infrastructure-heavy features, the team delivered a working system instead of an unfinished collection of partial subsystems.

19.2 Value of Layered Architecture

The layered Spring Boot architecture proved highly effective. It allowed the team to separate routing, business logic, and persistence in a way that made the application understandable and testable. This is a useful lesson for future projects: clean structure creates room for change.

19.3 Domain Logic Before Infrastructure

The final MVP demonstrates the advantage of building **domain truth first**. Before payments, mobile apps, QR scanning, or cloud scaling, the project needed to answer simpler but fundamental questions:

- how do users register?
- how are memberships represented?
- when is a check-in allowed?
- how is history recorded?

Once these truths exist in code, more advanced features become easier to justify and implement.

19.4 Testing as a Confidence Tool

The test suite was valuable not only for correctness but for communication. A working test suite gives external reviewers confidence that the application is not merely a visual prototype. In academic settings, automated tests also improve the credibility of design claims.

19.5 Honest Documentation Matters

A final lesson is the importance of distinguishing between **planned** and **implemented** scope. Good project reporting does not hide reductions in ambition; it explains them clearly and connects them to constraints and engineering decisions. This report therefore treats the differences between the early vision and the final repository as an expected part of the development process.

20. Conclusion

The OnePass repository contains a complete and working MVP of a multi-gym membership management platform. The final system demonstrates a clear layered architecture, secure password handling using BCrypt, session-aware route protection, business rules for membership access, seeded demo readiness, and automated tests that all pass successfully.

From a business perspective, the project remains grounded in a meaningful problem: the fragmentation of gym access in . The analysis work from Progress Report 1 established a compelling rationale for a unified multi-gym subscription model, and the design work from Progress Report 2 outlined an architecture suitable for a scalable future platform. The implemented MVP does not deliver every originally planned feature, but it successfully captures the platform's essential operational logic and provides a strong proof-of-concept.

From a software engineering perspective, the project demonstrates several strengths: modular design, clear requirement traceability, basic but appropriate security practices, useful automated testing, and a product that is easy to run and demonstrate locally. The main deferred areas—payments, partner workflows, QR-based entry, production persistence, and cloud deployment—define a realistic roadmap rather than a failure of execution.

In summary, OnePass should be viewed as a successful academic MVP that transforms an ambitious product concept into a functioning software system. It proves that the underlying domain model is implementable, that the core user journey is coherent, and that the project is well-positioned for future expansion into a more fully featured and production-ready fitness access platform.

21. References

Amazon Web Services. (2025). *AWS Canadian Regions and Compliance*.

<https://aws.amazon.com/compliance/-data-residency/>

Auth0. (2025). *Identity and Access Management Platform*. <https://auth0.com/>

Bause, K., Radimersky, A., Iwanicki, M., & Albers, A. (2014). Feasibility studies in the product development process. *Procedia CIRP*, 21, 473–478.

Payments Association. (2024). *Canadian payment compliance and security standards*.

<https://www.payments.ca>

ClassPass. (2025). *How ClassPass works for businesses*. <https://classpass.com/partners/how-it-works>

Deutsches Institut für Normung. (2009). *DIN 69901-2: Projektmanagement – Projektmanagementsysteme – Teil 2: Prozesse, Prozessmodell*. Beuth Verlag.

Fitness Avenue. (2025a). *Canadian gym membership statistics 2025*.

<https://www.fitnessavenue.ca/blogs/post/gym-membership-statistics->

Fitness Avenue. (2025b). *Average gym membership cost in : 2025 pricing guide*.

<https://www.fitnessavenue.ca/blogs/post/average-gym-membership-cost->

Health & Fitness Association. (2025). *'s fitness industry and the global context: An HFA global report summary*. <https://ficdn.ca/en/s-fitness-industry-and-the-global-context-an-hfa-global-report-summary>

IBISWorld. (2025). *Gym, health & fitness clubs in (Industry Report 71394CA)*.

<https://www.ibisworld.com/industry/gym-health-fitness-clubs/1655>

IEEE. (2011). *IEEE 29148–2011: Systems and software engineering — Life cycle processes — Requirements engineering*. Institute of Electrical and Electronics Engineers.

ISO/IEC. (2019). *ISO/IEC 25010:2019 — Systems and software quality requirements and evaluation (SQuaRE) — System and software quality models*. International Organization for Standardization.

Nielsen, J. (2012). *Usability engineering*. Morgan Kaufmann.

Office of the Privacy Commissioner of Canada. (2024). *Personal Information Protection and Electronic Documents Act (PIPEDA)*. <https://www.priv.gc.ca/en/privacy-topics/privacy-laws-in-canada/the-personal-information-protection-and-electronic-documents-act-pipeda/>

OWASP Foundation. (2024). *OWASP Top 10: The ten most critical web application security risks*. <https://owasp.org>

Palvia, P., & Palvia, S. (1988). The feasibility study in information systems: An analysis of criteria and contents. *Information & Management*, 14(5), 211–224.

PCI Security Standards Council. (2024). *PCI DSS Quick Reference Guide*. <https://www.pcisecuritystandards.org/>

Poppendieck, M., & Poppendieck, T. (2013). *Lean software development: An agile toolkit*. Addison-Wesley.

React. (2025). *React Documentation*. <https://react.dev/>

React Native. (2025). *React Native Documentation*. <https://reactnative.dev/>

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.

Stripe. (2025). *Stripe Pricing and Fees*. <https://stripe.com/en-ca/pricing>

Trustpilot. (n.d.-a). *ClassPass reviews*. Retrieved January 20, 2026, from <https://www.trustpilot.com/review/classpass.com>

Trustpilot. (n.d.-b). *Wellhub (Gympass) reviews*. Retrieved January 20, 2026, from <https://www.trustpilot.com/review/gympass.com>

Wellhub. (2025). *Wellhub for Companies*. <https://wellhub.com/en-ca/companies/>

Wiegers, K., & Beatty, J. (2013). *Software requirements* (3rd ed.). Microsoft Press.

Yahoo Health. (2025). *From Pilates to Parkour, ClassPass reports the most popular workouts of 2025*. <https://health.yahoo.com/wellness/fitness/exercise/articles/pilates-parkour-classpass-reports-most-190014628.html>

Yard Athletics. (n.d.). *How much does a gym membership cost?* Retrieved January 20, 2026, from <https://yardathletics.ca/how-much-does-a-gym-membership-cost>

Internal source: OnePassCanada repository inspection, configuration review, template review, and Maven test execution completed April 12, 2026.

22. Appendices

Appendix A — Review Checklist

Item	Included	Notes
Source code review	Yes	Based on repository inspection
Working software product	Yes	Spring Boot application
Business case	Yes	Sections 2–4
Feasibility study	Yes	Section 5
Requirements analysis	Yes	Section 6
Design goals and architecture	Yes	Sections 7–9
Database design	Yes	Section 10
Interface design	Yes	Section 11
UML / diagrams	Yes	Section 12
Implemented product documentation	Yes	Section 13
Security documentation	Yes	Section 14
Testing design and results	Yes	Section 15
Deployment instructions	Yes	Section 16
KPI evaluation	Yes	Section 17
Risks and roadmap	Yes	Section 18
Final conclusion	Yes	Section 20

Appendix B — Requirement Traceability Snapshot

Requirement	Implemented Module / Evidence
User registration	<code>AuthController</code> , <code>UserService</code> , <code>register.html</code>
Login/logout	<code>AuthController</code> , <code>login.html</code>
Membership selection	<code>MembershipType</code> , <code>register.html</code> , <code>membership.html</code>
Check-in	<code>CheckInController</code> , <code>CheckInService</code> , <code>checkin.html</code>
Cooldown rule	<code>CheckInService.checkIn()</code>
Daily limit	<code>MembershipType</code> , <code>CheckInService</code>
Visit history	<code>HistoryController</code> , <code>history.html</code> , <code>CheckInRepository</code>
Profile update	<code>AccountController.updateProfile()</code>
Password change	<code>AccountController.changePassword()</code> , <code>UserService.changePassword()</code>
Membership change	<code>AccountController.changeMembership()</code> , <code>UserService.changeMembership()</code>
Admin dashboard	<code>AdminController</code> , <code>admin.html</code>

Appendix C — Route Inventory

Route	Method	Purpose
/	GET	Redirect to home or login
/login	GET	Show login page
/login	POST	Authenticate user
/register	GET	Show registration page
/register	POST	Create account
/logout	GET	Invalidate session
/home	GET	Show dashboard
/checkin	GET	Show gyms
/checkin	POST	Submit check-in
/history	GET	Show visit history
/account	GET	Show account page
/account/update	POST	Update name
/account/password	POST	Change password
/membership	GET	Show membership page
/membership/change	POST	Change tier
/admin	GET	Show admin dashboard

Appendix D — Suggested Formatting for Submission

To achieve the requested **70–90 page final report** when exported to Word or PDF, the following formatting is recommended:

- Font: Times New Roman or Calibri, 12 pt
- Line spacing: 1.5
- Margins: 1 inch
- Include page breaks before major sections
- Convert textual diagrams into figure layouts where possible
- Add auto-generated table of contents in Word

With page breaks, diagrams, tables, and appendices, this document is intended to scale into the requested page range.

Appendix E — Glossary

Term	Meaning
MVP	Minimum Viable Product
MVC	Model–View–Controller
JPA	Java Persistence API
H2	Lightweight Java relational database
BCrypt	Password hashing algorithm
ARPU	Average Revenue Per User
ARR	Annual Recurring Revenue
RBAC	Role-Based Access Control
PIPEDA	Canadian privacy legislation
ORM	Object-Relational Mapping

Appendix F — Detailed Screen-by-Screen Walkthrough

The following appendix expands the interface-design discussion by documenting each screen in a more implementation-oriented way. This level of detail is useful for formal final submission because it shows not only what pages exist, but how each one contributes to the system's functional behavior, usability goals, and future extensibility.

F.1 Login Screen

The login screen is the primary entry point for returning users. It is intentionally minimal so that a user can authenticate with as little friction as possible. The screen contains an email field, a password field, a submit button, and a text link to the registration page. It also displays an inline error alert when authentication fails and includes a demo credential hint for presentation or grading convenience.

From a software engineering perspective, the login page demonstrates a clean division of responsibility. The template collects credentials, the controller forwards them to the service layer, and the service layer performs credential verification against the persisted user record using BCrypt. If authentication succeeds, the controller populates the session with the user's identity and role; if it fails, the page is reloaded with a flash error message. This means the screen is not merely cosmetic; it is the visible front end of a secure credential-handling workflow.

From a usability perspective, the page benefits from a highly focused layout. It avoids distracting secondary actions and keeps the number of choices low, which is especially appropriate for an authentication screen. For future versions, the login screen could be extended with password recovery, account lockout messaging, support contact information, and session-remembering options.

F.2 Registration Screen

The registration screen supports new-user onboarding. It requests the minimum information required by the MVP: full name, email address, password, and desired membership tier. This is simpler than the originally proposed commercial platform, which envisioned broader personal-profile data and payment information, but it is appropriate for the implemented phase because it keeps onboarding lightweight while still capturing the information needed to test membership logic.

The registration form has strong educational value because it illustrates multi-layer validation. The interface marks fields as required, but the service layer performs the more important checks, including blank-name prevention, email-format validation, password-length enforcement, and

duplicate-email detection. If all checks pass, the password is hashed, the user is stored, and the session is created immediately. This auto-login behavior reduces friction and helps demonstrate a complete onboarding journey in a single session.

From a future-development standpoint, this page could later be extended to collect billing information, date of birth, address, emergency contact details, health disclaimers, or privacy preferences. The current MVP keeps only the most essential fields, which is consistent with the principle of building a thin vertical slice before introducing additional data complexity.

F.3 Home Dashboard Screen

The home dashboard is the application's central user view and arguably the most important screen in the MVP. It provides a concise snapshot of the user's state in the system, including membership status, daily visit usage, total historical check-ins, cost-per-visit estimate, and balance. It also provides direct access to the most important next-step actions through the "Check In Now" and "View History" buttons.

This screen is valuable because it translates internal system data into user-facing meaning. Membership information is displayed in a visually prominent card, reinforcing the user's current plan and daily entitlement. The statistics grid conveys recent and cumulative engagement, while the cost-per-visit calculation helps connect product usage with the platform's economic value proposition. Even though the MVP does not include actual billing workflows, this dashboard element helps illustrate how subscription value can be communicated to end users.

In terms of design quality, the dashboard is intentionally compact and readable. It avoids excessive data density while still surfacing the most important indicators. In future versions, the home page could include recent gym activity, personalized gym recommendations, renewal notices, community challenges, or plan-upgrade recommendations based on usage patterns.

F.4 Check-In Screen

The check-in screen operationalizes the core business value of the application. It presents gyms in a grid of cards, each showing the gym name, location, and a button for submitting a check-in request. The interface is intentionally simple because the logic behind the screen is more important than the page itself: when the user clicks the check-in button, the system must validate rules and persist an outcome.

This page provides a useful example of how a simple interface can still represent a meaningful domain process. Behind the form submission, the system performs user lookup, gym lookup, same-gym cooldown validation, daily-limit counting, and status assignment. The screen then communicates the result through flash messages displayed on the redirected page. This makes the page highly suitable for demonstration because one interaction shows both the front-end experience and the back-end business logic.

Future iterations of this screen could include search, sort, map-based gym selection, amenity filters, operating hours, occupancy indicators, QR code generation, and location-based suggestions. For the MVP, the current form is intentionally basic so that the business rule engine remains the primary point of emphasis.

F.5 History Screen

The history screen is the user's audit trail of prior check-ins. It displays total check-ins at the top and then lists each historical record in reverse chronological order. Each item shows the gym name, timestamp, and status. Successful and warning outcomes are visually differentiated, which is useful because it preserves the meaning of edge-case events such as "Too Soon" or "Limit Reached."

This page is important in both product and engineering terms. Product-wise, it supports transparency and self-monitoring: the user can review where and when visits occurred. Engineering-wise, it demonstrates the value of persistent event recording. Because the system saves check-in attempts even when they do not result in a standard success, the history view becomes a more complete record of system behavior.

The current history page does not include filtering, searching, exporting, or pagination. These features were anticipated in earlier planning documents and remain logical future improvements. If the platform scaled beyond a course MVP, the history screen would likely evolve into a richer analytics and reporting surface for both users and administrators.

F.6 Account Screen

The account screen combines profile information, summary statistics, and account-management actions. It shows the user's membership tier, total check-ins, and current balance at the top. It then displays a profile-information section, a form for updating the name, and a form for changing the password. Links to membership management and logout complete the page.

This screen is significant because it reflects a mature understanding of account management within a small application. Instead of scattering account settings across multiple disconnected pages, the design groups related responsibilities in a clear and card-based layout. The use of discrete sections also makes the page easier to maintain and extend.

From an implementation perspective, the page demonstrates several distinct flows: data display, simple profile mutation, and credential change with secure current-password verification. The separation of these actions into different controller methods and service operations is good engineering practice because it avoids overloaded endpoints and keeps validation logic focused.

F.7 Membership Screen

The membership screen allows the user to compare all available tiers and switch to a different one. The current plan is highlighted visually, while alternative plans are presented with their monthly fee and daily visit entitlement. This design provides immediate side-by-side comparison without forcing the user to navigate elsewhere for pricing information.

This page is a strong example of how the system's domain model is exposed to the user. The `MembershipType` enum becomes both a technical construct and a visible product element. When a user changes tier, the platform updates the membership type and corresponding balance field, providing a lightweight simulation of plan management.

In a commercial system, this screen would likely become part of a more complex subscription workflow involving proration, billing-cycle alignment, receipts, cancellation policies, payment methods, and renewal terms. In the MVP, it functions as a controlled environment for plan-change behavior and illustrates how product rules can be surfaced in a usable form.

F.8 Admin Dashboard Screen

The admin screen provides the system operator with a summary of total users and total check-ins, followed by a table of all users. The table displays user ID, name, email, membership tier, role, balance, and member-since date. This screen is intentionally modest, but it establishes the concept of role-specific administrative oversight.

The admin view is valuable because it demonstrates two important engineering ideas: role-aware routing and platform-level visibility. The page can only be reached by a session whose role is `ADMIN`, and the controller pulls together both user-level data and system-wide metrics. Even in

this minimal form, it shows how operational insight can be layered onto the same domain model used for end-user workflows.

In later iterations, this dashboard could be expanded with charts, trend analysis, active-user counts, flagged events, gym utilization metrics, financial summaries, and export capabilities. The current implementation is sufficient for the MVP because it proves the platform can distinguish between ordinary users and privileged operators.

Appendix G — Detailed Technical Documentation by Module

This appendix expands the technical summary in the main body of the report by documenting the implementation responsibilities of the main classes and packages in greater depth. It is intended to support final-project review by showing how the repository is organized into understandable and traceable modules.

G.1 Application Bootstrap and Configuration

The `OnePassCanadaApplication` class is the standard Spring Boot entry point. Its responsibility is intentionally small: it boots the application context and hands control to the framework. Although this class contains minimal code, it plays an important architectural role by serving as the root of component scanning and auto-configuration.

The runtime configuration is primarily located in `application.properties`. This file defines the application name, H2 datasource settings, Hibernate DDL behavior, SQL logging, H2 console exposure, and server port. These settings reinforce that the project is optimized for local academic demonstration rather than hosted persistence.

G.2 DataLoader

`DataLoader` is one of the most practical classes in the repository from a demonstration standpoint. It implements `CommandLineRunner` and seeds gyms plus demo accounts at startup. This means the application always begins in a usable state without requiring manual SQL inserts or registration steps.

The class also demonstrates a responsible approach to seed logic by checking for user existence before inserting the demo accounts. It hashes the default passwords with BCrypt and assigns the admin role to the administrator account. This is important because it means demo setup follows the same security pattern as regular user creation.

G.3 AuthController

`AuthController` manages four closely related responsibilities: root redirection, login rendering/submission, registration rendering/submission, and logout. It acts as the bridge between public-facing authentication forms and the domain logic implemented in `UserService` .

This controller is structurally strong because it keeps business logic out of the route handlers. During login, it delegates authentication to the service layer and only concerns itself with session initialization and redirect behavior. During registration, it catches validation-related exceptions and returns the user to the appropriate page with an error message. The controller therefore serves as a good example of how Spring MVC controllers should remain thin and user-flow oriented.

G.4 HomeController

`HomeController` is responsible for the authenticated dashboard experience. It validates that a user session exists, loads the corresponding user, invalidates stale sessions if the user cannot be found, and calculates dashboard metrics through `CheckInService` .

This controller is more than a simple page renderer because it assembles information from multiple sources into a coherent user summary. It fetches today's visit count, total visit count, daily limit, and effective cost per visit. By concentrating these computations here rather than in the template, the application preserves a clean separation between presentation and business preparation.

G.5 CheckInController

`CheckInController` is the main interaction point for the system's core use case. The GET route loads available gyms and displays the selection page, while the POST route delegates the actual decision-making to `CheckInService` .

The controller's use of differentiated flash messages is particularly appropriate because it maps domain outcomes into user-readable feedback. A successful check-in, a same-gym cooldown event, and a daily-limit event each produce distinct messaging. This improves usability without forcing the controller to implement the business rules itself.

G.6 HistoryController

`HistoryController` has a focused and well-defined role: it verifies the session, retrieves the current user's check-in history, and passes both the list and its size to the template. This simplicity

is a strength rather than a weakness because it shows good layering. The controller knows just enough to prepare the screen and no more.

In future versions, this controller would likely grow to support filtering parameters, date ranges, gym search, and pagination. The current minimal design nevertheless provides a stable base for such additions.

G.7 AccountController

`AccountController` is one of the richest controllers in the application because it handles several related but distinct account-management workflows: displaying account details, updating the user's name, changing the password, viewing membership options, and changing the membership tier.

Even though it supports several routes, the controller remains readable because each method is strongly scoped. The implementation pattern is consistent: validate the session, load the current user where necessary, delegate to the service layer, and return with success or error feedback. This is good evidence that the project maintained structural discipline even as functional scope expanded.

G.8 AdminController

`AdminController` demonstrates role-aware access control in the repository. It first checks for an authenticated session and then checks that the stored role is `ADMIN`. Only then does it load user data and total system check-ins for the admin template.

The controller is intentionally simple, but it establishes a pattern that could be extended into more robust role-based administration later. Its presence also reinforces that the application is not purely consumer-facing; it acknowledges the operational needs of a platform owner or administrator.

G.9 UserService

`UserService` is the backbone of account-related domain logic. It manages registration, authentication, lookup by email, lookup by ID, profile updates, password change, membership change, and full user retrieval for admin use.

The registration method is especially important because it combines validation, duplicate-prevention logic, password hashing, and account creation. The password-change method is also

well-constructed because it verifies the current password before applying the new one. These service-level controls are essential because they prevent critical business and security logic from depending solely on controller behavior or client-side validation.

G.10 CheckInService

`CheckInService` contains the most distinctive business rules in the system. It validates whether the user exists, whether the gym exists, whether the same gym has been visited too recently, and whether the user has reached the daily limit for their membership tier. It then records a `CheckIn` entity with the correct status.

From a software engineering perspective, this class is central to the project's credibility because it translates the product concept into enforceable logic. It is also the clearest example of why service-layer concentration of domain rules matters: if the rules were scattered across templates or controllers, correctness and maintainability would suffer immediately.

G.11 GymService

`GymService` is intentionally small, containing gym retrieval behavior. Its modest scope is appropriate because the MVP treats gyms as mostly static seed data. Even so, the presence of a dedicated service is architecturally useful because it preserves consistency in the layering model and leaves room for future gym-related rules, such as filtering, availability checks, partner status, or admin-managed updates.

G.12 UserRepository

`UserRepository` extends `JpaRepository` and provides lookup by email plus email-existence checks. These methods are sufficient for the current domain because authentication and duplicate prevention both depend on email identity.

In future versions, this repository might also support role-specific queries, account-status filtering, or reporting projections. The current form remains intentionally concise and aligned to the MVP.

G.13 GymRepository

`GymRepository` relies entirely on default JPA CRUD behavior. This is appropriate because gyms are currently loaded through seed data and displayed without complex query logic. The

repository's simplicity reflects a healthy engineering pattern: do not add custom data access behavior until a real use case requires it.

G.14 CheckInRepository

`CheckInRepository` is the richest repository in the project because it answers several core business questions. It supports ordered history retrieval, total count per user, daily count within a time range, and a same-gym cooldown existence check.

This repository is central to the platform because it transforms stored check-in data into decision-making input. Without these queries, the system could not enforce plan rules or provide historical visibility.

G.15 User Model

The `User` entity is designed to support both product identity and administrative visibility. Its fields go beyond login data by including membership type, balance, creation date, and role. This makes it suitable for both end-user flows and back-office display.

The role default of `USER` is practical, and the explicit `ADMIN` assignment in seed data makes administrative testing straightforward. The model remains lightweight without being under-specified.

G.16 Gym Model

The `Gym` entity contains only name and location in the MVP. This is enough to support selection and display, while keeping the schema small. It also makes future extension relatively easy, since new attributes such as amenities, opening hours, occupancy, or partner status could be added without redesigning the entire domain.

G.17 CheckIn Model

The `CheckIn` entity represents the most important operational event in the platform. It ties a user to a gym at a timestamp and records the system's decision in the form of a status string. In many ways, this entity is the core event log of the application.

Because the project retains non-successful results such as `Too Soon` and `Limit Reached`, the model is more analytically useful than a simple success-only visit counter. That choice would be valuable later for fraud analysis, support, or billing reconciliation.

G.18 MembershipType Enum

The `MembershipType` enum is a compact but elegant implementation of business policy. Instead of scattering pricing and limit information across configuration files or conditionals, the application defines them as properties of the enum values themselves. This makes the code easier to read and reduces the chance of inconsistency between pages or services.

Appendix H — Planned vs Implemented Scope Reconciliation Matrix

This appendix provides a more detailed comparison between the original vision described across the earlier progress reports and the repository delivered as the final MVP. Its purpose is not only to show what changed, but to explain why the changes were reasonable within the constraints of the project.

H.1 Strategic Interpretation

The initial design and business materials described a platform that was closer to a startup-ready ecosystem than a course-scale prototype. It included web and mobile clients, partner onboarding, QR validation, billing, payout logic, and cloud deployment. All of these were defensible ideas from a long-term product perspective, but they significantly increased technical and operational scope.

The implemented repository deliberately narrowed the problem to what could be delivered with quality: a web-based MVP that validates account management, plan enforcement, gym selection, and visit recording. In software engineering terms, this represents a shift from broad horizontal ambition to a narrower vertical slice.

H.2 Comparison Matrix

Area	Early-Phase Expectation	Final Repository Status	Interpretation
Frontend architecture	React web frontend	Not implemented	Replaced by Thymeleaf server-rendered templates
Mobile app	React Native companion app	Not implemented	Deferred to later roadmap
Backend style	Node.js / NestJS style service layer	Not implemented	Replaced by Spring Boot MVC monolith
Database	PostgreSQL + Redis concept	Not implemented	Replaced by H2 in-memory DB for MVP simplicity
Cloud hosting	AWS-targeted deployment concept	Not implemented	Local runtime only
Authentication provider	Auth0 / Cognito discussed	Not implemented	Replaced by manual session authentication
Payment integration	Stripe and subscriptions envisioned	Not implemented	Deferred
Partner portal	Planned	Not implemented	Deferred
QR check-in	Planned and emphasized	Not implemented	Deferred
User registration	Planned	Implemented	Simplified and working
Membership tiers	Planned	Implemented	Tier rules encoded in enum
Gym listing	Planned	Implemented	Seeded list
Check-in logic	Planned	Implemented	Core business rules delivered
Visit history	Planned	Implemented	Basic but functional

Area	Early-Phase Expectation	Final Repository Status	Interpretation
Profile management	Planned	Implemented	Name/password update delivered
Admin dashboard	Planned	Implemented	Basic version delivered

H.3 Why the Scope Reduction Was Reasonable

The final scope reduction can be justified on technical, schedule, and educational grounds.

First, reducing the number of moving parts improves the odds of producing a stable final system. A split web/mobile/cloud/payment architecture would have required more integration work, more deployment coordination, and more failure points. For a student project, this can easily result in many partial subsystems and no reliable end-to-end workflow.

Second, the course assessment emphasizes software engineering process, traceability, design, and demonstrable implementation. A cohesive local MVP often serves those goals better than a partially integrated multi-platform ecosystem.

Third, the chosen MVP still preserves the essence of the original concept. Users still create accounts, select plans, view gyms, check in, and review their history. The admin still gains system-level visibility. In other words, the system did not abandon the problem; it postponed surrounding infrastructure.

H.4 Scope Reduction as an Engineering Success

In professional software delivery, controlled scope reduction is frequently a sign of maturity rather than compromise. Teams that insist on delivering every planned feature under tight constraints often produce unstable or untestable systems. Teams that identify the essential product slice and deliver it with quality usually create a better foundation for future growth.

The OnePassCanada repository fits the second pattern. It is smaller than the original vision, but it is coherent, testable, and extensible. That makes the final result stronger than a superficially broader but unreliable submission would have been.

H.5 What the Current Repository Proves

The final repository successfully proves that:

- the domain can be modeled clearly in code;
- the primary user journey is valid and demonstrable;
- the business rules can be enforced deterministically;
- the application is maintainable and testable;
- the concept is suitable for iterative extension.

H.6 What the Repository Does Not Yet Prove

At the same time, the repository does not yet prove:

- commercial payment viability;
- partner-gym operational integration;
- real-world fraud resistance;
- production infrastructure readiness;
- mobile user adoption patterns;
- at-scale performance behavior.

This distinction is important because it clarifies the exact maturity level of the project: it is a **working MVP**, not a production launch candidate.

Appendix I — Representative Acceptance and Test Catalog

The following appendix expands on the testing section by describing representative acceptance scenarios that the implemented system should satisfy. These cases combine business expectations and technical behavior. Some are directly covered by automated tests, while others are stated as manual acceptance checks appropriate for final review or demonstration.

I.1 Registration Acceptance Cases

ID	Scenario	Preconditions	Action	Expected Result
REG-01	Register with valid data	Email does not already exist	Submit valid name, email, password, and membership tier	Account is created, password is hashed, user is redirected to home
REG-02	Register with blank name	None	Submit blank or whitespace-only name	Registration rejected with validation message
REG-03	Register with invalid email	None	Submit malformed email value	Registration rejected with validation message
REG-04	Register with short password	None	Submit password under 6 characters	Registration rejected with validation message
REG-05	Register with duplicate email	Existing user present	Submit already-used email	Registration rejected with duplicate-email error

I.2 Authentication Acceptance Cases

ID	Scenario	Preconditions	Action	Expected Result
AUTH-01	Login with valid credentials	Account exists	Submit correct email and password	Session created; redirect to home
AUTH-02	Login with wrong password	Account exists	Submit valid email with invalid password	Authentication fails; login page shows error
AUTH-03	Login with unknown email	No such user	Submit unknown email	Authentication fails; login page shows error
AUTH-04	Logout	Active session exists	Navigate to logout route	Session invalidated; redirect to login

I.3 Home Dashboard Acceptance Cases

ID	Scenario	Preconditions	Action	Expected Result
HOME-01	View home with valid session	User logged in	Open /home	Dashboard renders with user data and stats
HOME-02	View home without session	No login	Open /home	Redirect to /login
HOME-03	Session contains stale user ID	Session exists, user missing	Open /home	Session invalidated; redirect to /login

I.4 Check-In Acceptance Cases

ID	Scenario	Preconditions	Action	Expected Result
CHK-01	Successful check-in under daily limit	User logged in; below tier limit	Submit gym ID	Check-in saved with Successful ; user redirected to history
CHK-02	Repeat check-in too soon at same gym	User checked into same gym within 30 minutes	Submit same gym ID	Check-in saved with Too Soon ; warning shown
CHK-03	Daily limit reached	User already at tier maximum for day	Submit any gym ID	Check-in saved with Limit Reached ; warning shown
CHK-04	Unauthorized check-in attempt	No session	Submit POST to <code>/checkin</code>	Redirect to login
CHK-05	View check-in page	Logged-in user	Open <code>/checkin</code>	Gym list displays correctly

I.5 History Acceptance Cases

ID	Scenario	Preconditions	Action	Expected Result
HIS-01	View history with records	Logged-in user with prior check-ins	Open <code>/history</code>	Records displayed in reverse chronological order
HIS-02	View history with no records	Logged-in user with no check-ins	Open <code>/history</code>	Empty-state message displayed
HIS-03	Unauthorized history access	No session	Open <code>/history</code>	Redirect to login

I.6 Account and Membership Acceptance Cases

ID	Scenario	Preconditions	Action	Expected Result
ACC-01	View account page	Logged-in user	Open /account	Account info and forms render
ACC-02	Update profile name successfully	Logged-in user	Submit non-blank name	User name updated and success shown
ACC-03	Update profile with blank name	Logged-in user	Submit blank name	Validation error shown
ACC-04	Change password correctly	Logged-in user knows current password	Submit current + valid new password	Password updated successfully
ACC-05	Change password with wrong current password	Logged-in user	Submit incorrect current password	Change rejected with error
MEM-01	View membership page	Logged-in user	Open /membership	Tier grid shown, current plan highlighted
MEM-02	Change membership	Logged-in user	Submit different tier	Membership updated and success shown

I.7 Admin Acceptance Cases

ID	Scenario	Preconditions	Action	Expected Result
ADM-01	Admin can access dashboard	Logged in as admin	Open <code>/admin</code>	Dashboard loads with system totals and user table
ADM-02	Non-admin blocked from dashboard	Logged in as regular user	Open <code>/admin</code>	Redirect to <code>/home</code>
ADM-03	Anonymous admin access blocked	No session	Open <code>/admin</code>	Redirect to <code>/login</code>

I.8 Deployment and Demo Acceptance Cases

ID	Scenario	Preconditions	Action	Expected Result
DEP-01	Application startup	Java 17 available	Run <code>mvnw.cmd spring-boot:run</code>	App starts on port 8080
DEP-02	H2 console access	App running	Open <code>/h2-console</code>	Console available with configured JDBC URL
DEP-03	Demo seed availability	App freshly started	Open app and log in with demo credentials	Demo user and admin are present

I.9 Test Coverage Interpretation

Not every acceptance case above is automated in the repository, but together they define a realistic acceptance baseline for the system. The existing automated test suite covers a strong subset of these cases, especially the service-layer logic and selected controller behaviors. Manual demonstration can reasonably cover the remaining high-level acceptance scenarios.

I.10 Recommended Future Acceptance Expansion

If the system evolves, the following additional acceptance suites would become necessary:

- payment initiation, renewal, and failure handling;

- QR generation and validation;
- partner-gym onboarding approval flows;
- financial reporting and payout accuracy;
- account recovery and rate-limited login behavior;
- search, filtering, and pagination behaviors;
- production-like performance thresholds.

Appendix J — Expanded Non-Functional Requirement Interpretation and Design Mapping

This appendix provides a deeper interpretation of the non-functional requirements that were discussed across the earlier reports and then partially realized in the final repository. The purpose of this appendix is to show that non-functional requirements were not treated as abstract theory; rather, they influenced implementation decisions, scope choices, and the evaluation criteria used in the final MVP.

J.1 Usability as a Design Driver

Usability was one of the strongest non-functional themes in the project from the beginning. The problem domain itself suggested that convenience is critical: if users are expected to choose a unified fitness pass over multiple fragmented memberships, the system experience must feel simpler, not more complex. This led to several practical design choices in the final implementation.

First, the application uses server-rendered pages with short and predictable user flows. This is especially visible in authentication, check-in, and membership-management journeys. For example, checking in does not require the user to navigate a complex multi-step wizard. Instead, the user sees a list of gyms, presses a single button, and receives a clear outcome. Likewise, the registration flow is intentionally short and avoids collecting excessive information in the MVP stage.

Second, the application relies on shared visual patterns. A common navigation bar, consistent card styling, repeated alert patterns, and similar form structures make the interface easier to learn. This is particularly important in an academic project because it demonstrates interface coherence even when advanced frontend frameworks are not used.

Third, the system uses user-oriented wording. Rather than exposing raw technical states, it provides messages such as "Check-in successful," "Please wait 30 minutes between visits to the

same gym,” and “Daily visit limit reached.” These messages help bridge the gap between internal business rules and user understanding.

J.2 Maintainability as a Structural Goal

Maintainability is arguably the most clearly realized non-functional requirement in the final repository. The project’s package structure separates controllers, services, repositories, models, configuration, templates, and static assets. This separation reduces coupling, improves readability, and makes it easier to reason about changes.

The service layer is especially important here. By locating domain logic in `UserService` and `CheckInService`, the project avoids duplicating rule logic across controllers or templates. This means that when a rule changes—such as a membership-tier limit or password policy—it can often be updated in one place. That is one of the most concrete indicators of maintainable design.

Maintainability is also improved by keeping the domain model relatively small. The project avoids premature complexity by focusing only on the entities required for the MVP: users, gyms, check-ins, and membership types. This makes the code easier to extend later because the foundational model is understandable and not overloaded with speculative attributes.

J.3 Portability and Demo Readiness

Portability was a major concern because the system needed to be demonstrated and reviewed in a predictable way. The final implementation addresses this through several deliberate choices. The use of the Maven Wrapper means reviewers do not need a global Maven installation. The use of an H2 in-memory database means reviewers do not need to configure MySQL, PostgreSQL, or other external services. The seed loader ensures that the application is useful immediately after startup.

These choices are not merely conveniences; they directly increase the project’s practical value as an academic deliverable. A project can be technically impressive but still difficult to review if the setup process is fragile. By contrast, OnePassCanada emphasizes reproducibility. This improves the chances that the software can be demonstrated consistently across machines and environments.

J.4 Testability and Verification

Testability is a quality attribute that influences how a system should be structured long before tests are written. In this project, the presence of a clean service layer and thin controllers made

testing far easier. Unit tests target the service classes directly, while controller tests verify route behavior through MockMvc. This arrangement reflects strong software engineering discipline.

The final test results, which show 23 passing tests, support the claim that the application is more than a prototype. They also show that the team selected an architecture compatible with automated verification. If the project had relied heavily on tightly coupled controllers or unstructured logic, test creation would have been more difficult and confidence would have been lower.

J.5 Security as a Controlled MVP Concern

Security was treated with selective seriousness in the MVP. The team focused on the most critical account-level measures—secure password handling and protected routes—while deferring broader production-grade concerns such as MFA, password reset, CSRF hardening, and rate limiting.

This prioritization is defensible. In the context of a course-scale MVP, hashing passwords with BCrypt is a much more important first step than simulating an enterprise-grade security stack without the surrounding infrastructure to support it. Likewise, using server-side session checks is a better approach than leaving pages unprotected. The security posture is therefore limited but sensible.

J.6 Reliability and Data Integrity

Reliability in the MVP is mixed. On one hand, the application reliably enforces its check-in rules and records outcomes consistently. On the other hand, because the database is in-memory and recreated on each run, the system does not provide durable persistence. This creates an important distinction between **runtime reliability** and **operational durability**.

The system is reliable within a single execution session. It will preserve user and check-in data while the app is running, it will enforce rules consistently, and it will allow repeated demonstrations without logic drift. However, it is not durable across restarts, which is why it should be described as demo-ready rather than production-ready. This is an important nuance for final documentation.

J.7 Scalability and Performance Expectations

The earlier reports discussed ambitious performance and scalability expectations, including sub-second check-in validation and support for large concurrent user counts. The final MVP does not validate those at scale, and it is important to say so explicitly. However, the final architecture still supports reasonable local performance for a small user base because the application is lightweight, the database is local, and the page-rendering model is uncomplicated.

The more important lesson is that scalability should not be claimed without evidence. In this final report, the project acknowledges that large-scale concurrency and nationwide deployment remain future concerns. This honest framing is preferable to making unsupported claims about performance readiness.

J.8 Compatibility and Browser Accessibility

Because the application is a web application using standard HTML and CSS, it is broadly compatible with modern browsers. This is an advantage over native-only approaches because it lowers the barrier to testing and access. The earlier concept also included mobile-native ideas, but the final repository shows that a browser-based MVP can still deliver meaningful value while remaining highly accessible.

Although no formal accessibility audit was documented, the structure of the interface—forms, labeled inputs, readable layout, and server-rendered HTML—creates a reasonable foundation for later accessibility improvements. A future version should still add explicit accessibility review, keyboard-navigation validation, and color-contrast analysis.

J.9 Non-Functional Requirement Mapping Matrix

Non-Functional Requirement	Intent	Repository Evidence	Degree of Fulfillment
Usability	Simple and clear user experience	Consistent navigation, alerts, short flows	Strong
Maintainability	Code should be easy to understand and modify	Layered MVC structure	Strong
Testability	Critical behaviors should be verifiable automatically	23 automated tests	Strong
Security	Protect accounts and sensitive actions	BCrypt, session checks, admin role checks	Moderate
Portability	Application should run easily on local machines	Maven Wrapper, H2 DB, seeded data	Strong
Reliability	Domain rules should behave consistently	Centralized check-in logic and persistence during runtime	Moderate
Scalability	System should support larger usage patterns	Not validated in repository	Weak / future
Performance	Fast response for core actions	Good local responsiveness, no benchmarks	Partial
Compatibility	Accessible from standard user environments	Browser-based app, standard HTML/CSS	Strong

J.10 Lessons from the Non-Functional View

Looking at the project through non-functional requirements clarifies why the final MVP is stronger than a superficially larger but more fragmented implementation would have been. The system emphasizes the qualities that were most achievable and most important for a course-scale

product: maintainability, usability, portability, and testability. More demanding qualities such as production scalability and commercial-grade security were not ignored, but deliberately deferred. This balanced treatment is itself a sign of thoughtful engineering.

Appendix K — Extended Business Process Narratives

This appendix documents the system's main business processes in a more narrative form than the UML diagrams in the main report. These narratives are useful because they show how product behavior, user intention, and system logic align in practice.

K.1 Account Registration Process Narrative

The registration process begins when a new user opens the registration page and enters a name, email address, password, and desired membership tier. At the interface level, the form appears straightforward and minimal. At the business level, however, this action represents several checks and state transitions.

The controller receives the submitted values and passes them into `UserService`. The service verifies that the name is not blank, that the email matches a valid format, that the password meets the minimum length, and that the email is not already associated with another account. If any of these conditions fail, the system rejects the request with an informative error. If they all succeed, the password is hashed with BCrypt, the account is saved, and the user is automatically logged in via the session.

From a business perspective, the process establishes a new member profile while assigning a plan and initial balance. From a user perspective, the process is short and low-friction. From a software perspective, it demonstrates a well-contained mutation of persistent system state.

K.2 Authentication Process Narrative

The authentication process begins when an existing user submits their email and password on the login page. The system must transform this credential submission into either a valid session or a denied access response.

The service layer first looks up the user by email. If the account does not exist, authentication fails. If it does exist, BCrypt checks whether the submitted password matches the stored hash. On success, the controller creates session state by storing the user ID, user name, and role. The user is

then redirected to the home dashboard. On failure, the user is returned to the login screen with an error message.

This process is important because it defines the boundary between anonymous and authenticated interaction. Every major page beyond login and registration relies on this boundary being enforced consistently.

K.3 Dashboard Access Process Narrative

The dashboard process begins when a logged-in user opens the home page. The system checks whether a valid `userId` exists in the session. If no session exists, the user is redirected to the login page. If a session exists, the system retrieves the user record and calculates summary metrics such as today's check-ins, total check-ins, current daily limit, and effective cost per visit.

This process is an example of how the application turns raw domain data into human-meaningful information. The business model of a multi-gym pass is more persuasive when the user can see their plan, their usage, and their historical engagement in one place. The dashboard is therefore both a navigational tool and a value-communication surface.

K.4 Gym Check-In Process Narrative

The gym check-in process is the heart of the application. When a user selects a gym and submits a check-in request, the system performs several validations. It confirms that the user exists, confirms that the gym exists, checks whether the user has already checked into that same gym within the last 30 minutes, counts the number of check-ins for the current day, and compares that number with the user's daily limit.

If the same-gym cooldown rule is violated, the system records the event with the status `Too Soon`. If the user is still under the daily limit, it records the event with the status `Successful`. If the daily limit has already been reached, it records the event with the status `Limit Reached`.

This process is noteworthy because it shows how a product promise—"membership tiers control access"—becomes enforceable system logic. It also illustrates why event persistence matters: even non-standard outcomes are saved and can later be displayed in the history view.

K.5 Visit History Process Narrative

The history process begins when the user opens the history page. The system uses the session identity to retrieve all check-ins associated with that user and orders them by descending

timestamp. The resulting list is rendered in a readable and status-aware format.

From a business perspective, this process provides transparency and accountability. From a technical perspective, it demonstrates how an event repository can be repurposed for user-facing reporting. The fact that history includes multiple status types also creates a fuller operational picture of system behavior.

K.6 Profile Update Process Narrative

The account-management process contains two especially important subflows: name update and password change. When a user changes their name, the system validates that the new name is not blank and then updates the persisted user record. When the user changes their password, the system verifies the current password first, validates the new password length, and only then stores a new BCrypt hash.

These processes show that even small account features should not be treated casually. Password change, in particular, is a security-sensitive action and is handled with appropriate validation logic.

K.7 Membership Change Process Narrative

The membership-change process begins on the membership page when the user selects a tier different from their current one. The system retrieves the user, updates the `membershipType`, updates the balance to match the tier fee, and saves the result. The user is then redirected back to the membership screen with success feedback.

This process is a simplified stand-in for what would become a more complex subscription workflow in a commercial platform. Even in its simplified form, it demonstrates an important product behavior: plans are not just displayed; they influence user state and future rule enforcement.

K.8 Admin Monitoring Process Narrative

The admin process begins when an authenticated session with the `ADMIN` role requests the dashboard. The controller verifies the role, then retrieves a full user list and total system check-in count. This information is rendered into summary cards and a user table.

This process is operationally important because it shows how one application can support more than one user perspective. The admin is not using the system as a member; the admin is

monitoring the system as a platform. Even though the view is basic, it captures an important conceptual layer in the overall design.

K.9 Failed and Edge-Case Process Narrative

An important part of software engineering is documenting not only successful flows but also failure and edge-case behavior. The final MVP addresses several such cases explicitly. An unauthenticated user attempting to open protected pages is redirected to login. A stale session pointing to a missing user is invalidated. Duplicate email registration is rejected. Incorrect current password prevents password change. Cooldown or limit violations still create history records.

These behaviors matter because they show that the system is not designed around a single happy path. It accounts for the ordinary mistakes and unusual states that real users generate.

K.10 Business Process Reflection

Taken together, these business process narratives show that the final MVP is richer than a static academic demo. It embodies a recognizable operational model with identity, access, rules, exceptions, and reporting. This strengthens the overall credibility of the final deliverable.

Appendix L — Future Production Architecture Proposal

This appendix outlines how the current OnePassCanada MVP could evolve into a more production-ready system while preserving the domain logic already established in the repository. The purpose is not to redefine the final project retroactively, but to demonstrate architectural foresight and to connect the current implementation to the original long-term vision.

L.1 Evolution Strategy Overview

The current repository should be viewed as a stable core rather than as a dead-end prototype. The most valuable aspect of the implementation is that the central domain concepts—users, gyms, check-ins, and membership types—are already modeled clearly. Future architecture work would therefore focus on scaling and externalizing capabilities rather than replacing the business foundation entirely.

L.2 Recommended Persistence Evolution

The first major production transition would be replacing the H2 in-memory database with a persistent relational database such as PostgreSQL. This would allow the platform to preserve users, gyms, and visit histories across restarts and would support more realistic operational workflows.

A migration framework such as Flyway or Liquibase should be introduced to manage schema evolution safely. This would prevent schema drift and would make it possible to version database changes alongside application code. Seed logic could remain, but demo data should be separated from production initialization routines.

L.3 Recommended Security Evolution

Spring Security should replace the current hand-rolled session approach. This would provide centralized route protection, login flow handling, CSRF defenses, password-policy configuration, and integration points for future roles or identity providers.

As the platform grows, account recovery and multi-factor authentication become more important. Administrators and partner users in particular should receive stronger authentication guarantees. Security logging and suspicious-login detection would also become valuable once the platform handles real payment or partner data.

L.4 Recommended Deployment Evolution

For production hosting, the application should be containerized and deployed using infrastructure that supports environment-based configuration, health checks, log aggregation, and monitoring. Even if the system remains a modular monolith initially, deploying it as a managed service with persistent storage and secure secrets handling would represent a major step forward.

Continuous integration and deployment pipelines would also be valuable. Automated testing on each commit, artifact packaging, and deployment promotion would reduce release risk and encourage disciplined change management.

L.5 Recommended API Strategy

If the product later reintroduces the original mobile-app vision, a dedicated API layer will be necessary. The current repository is server-rendered, which works well for the web MVP but is not an ideal foundation for native or third-party clients. One possible evolution path is to introduce REST endpoints gradually while continuing to support Thymeleaf pages for internal and demo workflows.

This hybrid period would allow the team to preserve existing value while exposing specific workflows—such as authentication, gym listing, and check-in—to external clients. Over time, the application could evolve into a more explicit backend-for-frontend model.

L.6 Recommended Gym and Partner Expansion

The current `Gym` model is intentionally minimal. A production version would likely extend it with business identifiers, operating hours, facility metadata, active/inactive partner status, payout rates, occupancy or capacity fields, and perhaps a relationship to partner administrators. A dedicated partner user model or role would also be needed.

On the interface side, this would support a partner dashboard, visitor insights, and gym-specific reporting. On the backend side, it would support partner onboarding, partner-level authorization, and eventual payout calculations.

L.7 Recommended Billing Evolution

Payment support should be introduced only after persistence, security, and event consistency are mature. Subscription billing creates new requirements: invoice generation, failed-payment handling, retry policy, receipt storage, tax treatment, cancellation policy, plan proration, and audit history.

The current repository already contains the beginnings of a pricing model through `MembershipType`, but a real billing system would likely require separate subscription entities, payment transaction records, billing-cycle tracking, and status fields such as active, paused, overdue, or cancelled.

L.8 Recommended Event and Analytics Evolution

The current `CheckIn` entity is already an event-like record and could become the basis for broader analytics. A future architecture might introduce reporting tables or derived metrics for:

- gym utilization trends;
- user retention patterns;
- membership-tier performance;
- fraud indicators;
- partner payout calculation.

If event volume became significant, asynchronous processing could be introduced for analytics, notifications, and downstream reconciliation. The current MVP does not need this complexity, but the conceptual path is clear.

L.9 Suggested Future Architecture Diagram

Client Layer

- Web client
- Mobile client

Application Layer

- Auth module
- Membership module
- Check-in module
- Admin module
- Partner module
- Billing module

Platform Services

- Notification service
- Reporting/analytics service
- Payment gateway integration

Data Layer

- PostgreSQL
- Object storage for reports/assets
- Cache layer (optional)

Operations Layer

- CI/CD pipeline
- Monitoring/logging
- Secret management

L.10 Why the MVP Still Matters in This Future View

It is important not to interpret future architecture proposals as evidence that the current MVP is insufficient. Rather, the MVP provides the domain and workflow baseline from which future architecture can grow. The most expensive kind of failure in early-stage software projects is not lacking cloud deployment; it is lacking a coherent model of the business process. OnePassCanada

does not have that problem. Its domain foundation is solid, and that makes future architectural evolution far more realistic.

L.11 Migration Path Summary Table

Current MVP Capability	Future Production Upgrade
H2 in-memory DB	PostgreSQL + migrations
Manual session auth	Spring Security + stronger policies
Server-rendered pages only	Hybrid or API-backed multi-client architecture
Static seeded gyms	Partner-managed persistent gym records
Enum-based simple plan logic	Subscription entities and billing engine
Local startup	Containerized cloud deployment
Basic admin dashboard	Analytics and operational reporting suite

L.12 Final Architectural Reflection

The most important architectural lesson is that product evolution does not always require discarding an MVP. When an MVP is cleanly structured and centered on accurate domain rules, it can serve as a durable stepping stone. OnePassCanada appears to fall into that category. Its current architecture is intentionally modest, but its internal clarity makes it a strong foundation for future work.